

Prosjektoppgave H25

Dokumentasjon

Akademisk tekst

Gruppe 8; Jimmy, Erica, Philip, Simen, Mohamed

21. november 2025
Halden



Innhold

Innhold	i
1 Innledning	1
1.1 Dagens løsning: Østfold kollektiv	1
1.2 Behovet for en ny løsning	1
1.3 Problemstilling	2
2 Problemforståelse og domenebeskrivelse	3
2.1 Brukere og behov	3
2.2 Personas	4
2.3 Scenarier	4
2.4 Brukerhistorier	5
3 Foreslått løsning (konseptnivå)	6
3.1 Overordnet konsept	6
3.2 Hvordan løsningen dekker behovene	6
3.3 Hovedkomponenter i systemet	7
3.4 Konseptuell arkitekturmodell	8
3.5 Oppsummering	8
4 Kravspesifikasjon	10
4.1 Funksjonelle krav	10

4.2	Ikke-funksjonelle krav	13
4.3	MVP Tekniske spesifikasjoner	14
4.4	Kravoversikt med estimer	15
4.5	Akseptansekriterier for MVP	16
5	Arbeidsprosess og organisering	17
5.1	Arbeidsmetodikk	17
5.2	Verktøy og teknisk infrastruktur	18
5.3	Oppgavefordeling og samarbeid	21
6	Sprint 1 – Planlegging og gjennomføring	23
6.1	Sprintmål	23
6.2	Sprint backlog	24
6.3	Kanban-board	24
6.4	Ansvarsfordeling	25
6.5	Sprint 1 – Retrospektiv	26
6.6	Datamodell (ER-diagram)	28
6.7	Oppsett av produksjonsserver	32
6.8	Oppsett av login-/registreringssystem	33
7	Sprint 2 – Planlegging og gjennomføring	35
7.1	Sprintmål	35
7.2	Sprint backlog	36
7.3	Kanban-board	36
7.4	Burndown-chart	37
7.5	Database og dummydata	38
7.6	Brukerautentisering og sikkerhet	42
7.7	Infrastruktur og deployment	44

7.8	Retrospekt Sprint 2	45
8	Sprint 3 – Planlegging og gjennomføring	47
8.1	Sprintmål	47
8.2	Planlegging av Sprint 3	47
8.3	Sprint backlog	48
8.4	Kanban-board	49
8.5	Burndown-chart	50
8.6	Databaseintergrasjon: Backend til PostgreSQL	50
8.7	Frontend - ruteplanlegging og nettside-layout	52
8.8	Retrospektiv	56
9	Sprint 4 – Planlegging og gjennomføring	58
9.1	Sprintmål	58
9.2	Planlegging av Sprint 4	58
9.3	Gjennomføring og oppgaver	59
9.4	Frontend – Gjennomført arbeid	59
9.5	Oppsummering	61
9.6	Forventet resultat av Sprint 4	61
9.7	Utfordringer og læringspunkter	61
9.8	Resultat og refleksjon	62
10	Status for prosjektet ved innlevering	63
10.1	Kravspesifikasjon	63
10.2	Design og modeller	63
10.3	Prototype og implementasjon	64
10.4	Testing	65
10.5	Organisering og arbeidsprosess	65

10.6 Oppsummering	66
11 Utviklingsprosess	67
11.1 Veien til produktet	67
11.2 Verktøy og metoder	70
11.3 Refleksjoner over utviklingsprosessen	72
12 Arkitektur	74
12.1 Systemarkitektur	74
12.2 Systemkomponenter	74
12.3 Avhengigheter og funksjonalitet	75
12.4 Begrensinger og antagelser	76
12.5 Deployment og arbeidsflyt	76
12.6 Oppsummering	76
13 Prototype	77
13.1 Prototype-rapport: Implementert funksjonalitet	77
13.2 Hvordan prototypen startes og brukes	78
13.3 Testing	78
13.4 Kjente svakheter og begrensninger	78
13.5 Sammenligning: Planlagt vs. implementert	79
14 Testing og validering	80
14.1 Kjøring av tester i IntelliJ IDEA	80
14.2 Teststrategi og tilnærming	81
14.3 Hvilke krav som er testet	81
14.4 Hvordan testene verifiserer funksjonaliteten	84
14.5 Hvordan systemet håndterer feil og uventede situasjoner	85
14.6 Testresultater og kvalitetssikring	87

14.7 Konklusjon	87
15 Bruken av kunstig intelligens	88
15.1 Verktøy brukt i prosjektet	88
16 Oppsummering og konklusjon	90
Referanseliste	91

Kapittel 1

Innledning

Formålet med denne kravspesifikasjonen er å dokumentere og definere de kravene som gjelder for utviklingen av en ny buss-app. Målet er å tilby et enklere og mer brukervennlig alternativ enn dagens løsning, *Østfold kollektiv*. Den nye appen skal fokusere på bedre funksjonalitet, tilgjengelighet, brukervennlighet og forståelse for brukernes behov.

1.1 Dagens løsning: Østfold kollektiv

Østfold kollektiv er en transport-app som brukes for å finne hvilken buss man skal ta for å komme frem til sin destinasjon. Appen har imidlertid flere svakheter som gjør at brukeropplevelsen oppleves som lite effektiv og lite tilgjengelig. Vi har identifisert følgende utfordringer:

- Brukergrensesnittet er lite intuitivt og oppleves som unødvendig komplisert.
- Appen gir ingen informasjon om forsinkelser i sanntid, noe som er kritisk for pålitelig kollektivtransport.

1.2 Behovet for en ny løsning

Behovet er dermed en app som vil gi brukeren:

- Enklere og mer tilgjengelig informasjon.
- Sanntidsdata som sikrer bedre punktlighet og forutsigbarhet.
- Enkel og rask navigasjon i appen.

1.3 Problemstilling

Oppgaven vi har fått er bred og kunne løses på mange måter. For å gi prosjektet en tydelig retning har vi valgt å spisse det inn mot en konkret problemstilling:

Hvordan kan vi utvikle en buss-app som gir mer brukervennlig tilgang til sanntidsinformasjon, ruteplanlegging og billettkjøp enn dagens løsning, Østfold kollektiv?

Denne problemstillingen fungerer som et styrende rammeverk for resten av rapporten. Alt videre arbeid med personas, scenarier, krav og estimering tar utgangspunkt i dette spørsmålet.

Kapittel 2

Problemforståelse og domenebeskrivelse

2.1 Brukere og behov

For å kunne sette krav til løsningen er det viktig å forstå hvem som skal bruke systemet. De ulike brukergruppene har forskjellige behov og ulik teknologisk kompetanse. Disse behovene må gjenspeiles i både funksjonelle og ikke-funksjonelle krav. Brukerne av systemet kan deles inn i tre hovedgrupper: kunder, administrator og utviklere.

- **Kunder:** Vanlige passasjerer som reiser med buss (skoleelever, pensjonister, studenter eller arbeidere) De har behov for enkel tilgang til bussruter, billettkjøp, rask informasjon om forsinkelser og oversikt over hvor bussen befinner seg i sanntid.
- **Administrator:** Ansatte i kollektivselskapet som er ansvarlige for å holde rutetider oppdatert, og sørge for at informasjonen i systemet er korrekt. For administratorene er det viktig at grensesnittet er intuitivt, lett å bruke og effektivt.
- **Utviklere:** Har ansvar for å vedlikeholde og videreutvikle systemet etter hvert som nye behov oppstår. De trenger gode verktøy for utrulling av oppdateringer, oppdaging av feil og testing av nye funksjoner.

Vi valgte disse tre brukergruppene fordi de representerer de viktigste aktørene i systemet: sluttbrukere som benytter appen daglig, administratorer som sørger for kvaliteten på dataene, og utviklere som skal vedlikeholde og videreutvikle løsningen. Ved å inkludere alle tre sikrer vi at kravene dekker både brukeropplevelse, driftsstøtte og tekniske behov.

2.2 Personas

For å forstå brukernes behov har vi laget fem personas som representerer ulike målgrupper. Personas representerer typiske brukere med ulik bakgrunn og kompetanse, og bidrar til å tydeliggjøre motivasjon og behov [4].

- **Philip (10)** – Skoleelev som tar buss til og fra skolen, særlig om vinteren. Han har begrenset teknologisk erfaring og ønsker enkel og rask informasjon om når bussen kommer, slik at han slipper å stå ute i kulden unødvendig.
- **Emma (24)** – Student som pendler daglig til høghskolen. Hun er vant til digitale verktøy, men opplever dagens app som lite intuitiv. Hun har behov for tydelig ruteplanlegging, sanntidsoppdateringer og en smidig reiseopplevelse.
- **Adam (35)** – Flyktning og småbarnsfar med begrensede norskkunnskaper. Han bruker buss for å følge barna til aktiviteter, men strever med språkbarrierer og uoversiktlige menyer. Han har behov for språkvalg og tydelig visuell navigasjon.
- **Berit (72)** – Pensjonist med lav digital kompetanse. Hun blir lett overveldet av komplekse menyer og små knapper. Hun trenger et brukervennlig grensesnitt med høy kontrast, store knapper og så få steg som mulig.
- **Lars (40)** – Administrator i kollektivselskapet. Han oppdaterer rutetider og rapporterer avvik. Han har behov for raske og effektive verktøy for databehandling, tydelige skjemaer og et driftsgrensesnitt som gjør rutineoppgaver enkle og feilfrie.

Vi har valgt å utvikle fem personas for å dekke ulike aldersgrupper, bakgrunner og teknologisk erfaring. Disse representerer både typiske sluttbrukere (barn, studenter, eldre og innvandrere) samt interne brukere (administratorer). Begrunnelsen er at variasjonen i personas gjør det mulig å avdekke et bredt spekter av behov, og gir oss en tydelig kobling mellom faktiske brukerutfordringer og kravene vi senere definerer.

2.3 Scenarier

Scenariene illustrerer hvordan brukerne opplever dagens system og hvilke problemer de møter. Slike korte narrativer er en effektiv måte å synliggjøre sammenhengen mellom brukernes mål og systemets funksjonalitet [4]. De fungerer som fortellinger som setter kravene i en kontekst, og viser hvordan brukerne ønsker å løse konkrete oppgaver.

Scenario 1 (Emma, student)

Emma åpner bussappen og søker etter rute fra Sarpsborg Bussterminal til Remmen Høghskole. Systemet foreslår buss 633 (Skjeberg–Halden) som den raskeste ruten, med en estimert reisetid på 40 minutter.

Men appen gir ingen informasjon om sanntidsforsinkelser, og Emma er usikker på om bussen faktisk er i rute. For å være sikker åpner hun Google Maps for å sjekke sanntid og bussens posisjon. Hun rekker til slutt bussen, men opplever det som tungvint å måtte bruke to ulike apper for å få komplett informasjon.

Scenario 2 (Adam, forelder og flyktning)

Adam åpner appen for å finne busser som barna kan ta til skolen. Han sliter fordi appen kun er på norsk, og han forstår ikke menyene godt nok. For å løse dette bruker Adam Google Translate for å oversette teksten i appen til sitt eget språk. Han klarer til slutt å finne riktig bussrute, men opplever prosessen som tungvint og tidkrevende. Han ønsker derfor at appen hadde hatt språkvalg direkte innebygd.

Scenariene bygger direkte på personas, og er laget for å validere at de identifiserte behovene er realistiske og relevante. Ved å knytte brukeropplevelser til konkrete situasjoner, som forsinkelser eller språkbarrierer, kan vi bedre forstå konsekvensene av dagens problemer og tydeliggjøre hvorfor nye funksjoner er nødvendige.

2.4 Brukerhistorier

Brukerhistorier hjelper oss å formulere behovene presist og kortfattet. Vi har brukt standardformatet: «*Som en [rolle] ønsker jeg å [handling] slik at jeg kan [mål]*», som en praktisk metode for å gjøre krav testbare og knytte dem direkte til brukerens behov [4]. Brukerhistoriene bygger på scenariene og gir grunnlag for å identifisere funksjonelle krav.

- Philip: Som skoleelev ønsker jeg å få beskjed om forsinkelser slik at jeg slipper å vente ute i kulden.
- Emma: Som ny student ønsker jeg at appen viser hvilke busser som går til høyskolen slik at jeg slipper å bruke taxi eller spørre tilfeldige folk om veien.
- Adam: Som flyktning ønsker jeg å bruke appen på mitt språk slik at jeg lettere kan finne bussruter.
- Berit: Som pensjonist ønsker jeg et enkelt grensesnitt med store knapper slik at jeg kan bruke appen uten teknisk kunnskap.
- Admin: Som admin ønsker jeg å oppdatere rutetider slik at kundene alltid får riktig informasjon.
- Utvikler: Som utvikler ønsker jeg å kjøre automatiserte tester slik at jeg raskt kan oppdage feil.

Brukerhistoriene fungerer som et bindeledd mellom scenarier og kravspesifikasjon. Ved å formulere behovene i dette formatet kan vi lettere validere at hvert krav faktisk svarer på et brukerbehov. Dette bidrar også til å prioritere kravene senere, siden vi kan vurdere nytte og teknisk kompleksitet basert på konkrete brukerhistorier.

Kapittel 3

Foreslått løsning (konseptnivå)

Den foreslåtte løsningen er et mer moderne og brukervennlig kollektivsystem. Løsningen er designet for å møte behovene til både passasjerer, administratorer og utviklere. Målet er å utvikle en bussapplikasjon som gir en mer pålitelig sanntidsinformasjon, enklere ruteplanlegging og mer oversiktlig billett-kjøp enn dagens løsning. Systemet følger en lagdelt og modulær arkitektur der brukergrensesnitt, backend-tjenester og datalagring, er tydelig separert. Dette gjør systemet skalerbart, vedlikeholdbart og enkelt å videreutvikle.

3.1 Overordnet konsept

Løsningen består av en mobilvennlig applikasjon der brukeren kan:

- planlegge reiser mellom to punkter
- se sanntidsinformasjon om forsinkelser og bussposisjoner
- kjøpe og fremvise digitale billetter

I tillegg tilbys et administrasjonsgrensesnitt for ansatte og administratorer, samt tekniske verktøy for utviklere som muliggjør videre drift og funksjonsutvidelser.

Systemet er bygd på et flerlagsprinsipp med presentasjonslag (frontend), tjenestelag (backend) og datalag (database og eksterne datakilder). Dette gjør at funksjoner kan utvikles uavhengig av hverandre samtidig som helheten opprettholdes.

3.2 Hvordan løsningen dekker behovene

Løsningen er direkte utviklet med utgangspunkt i personas, scenarier og brukerhistorier presentert tidligere:

- **Sanntidsinformasjon** dekker behovet til Philip og Emma, som trenger pålitelig informasjon for å unngå forsinkelser og usikkerhet.
- **Enkel navigasjon og tydelige visuelle elementer** hjelper Berit med lav teknologisk erfaring.
- **Flerspråklig støtte** gir Adam en bedre og mer inkluderende brukeropplevelse.
- **Administrasjonspanel** gir Lars mulighet til å oppdatere rutetider og registrere avvik raskt.
- **Strukturert og testbar backend** dekker utviklerens behov for driftssikkerhet, vedlikehold og automatiserte tester.

Løsningen er derfor ikke kun teknisk motivert, men rettet direkte i reelle brukerbehov og utfordringer fra dagens system.

3.3 Hovedkomponenter i systemet

Den konseptuelle løsningsarkitekturen består av følgende hovedkomponenter:

1. Brukergrensesnitt (Frontend)

Et enkelt og responsivt brukergrensesnitt tilgjengelig via mobile-first. Det består av:

- reiseplanlegger
- billettkjøp og billettvisning,
- brukerkonto og innstillinger.

Designet fokuserer på universell utforming, store klikkflater, god kontrast og et språktilpasset innhold.

2. API- og forretningslogikk (Backend)

Backend fungerer som systemets kjernelogikk og tilbyr:

- ruteplanlegging basert på databasen,
- håndtering av billetter og QR-koder,
- brukerautentisering og tilgangskontroll,
- administrativ funksjonalitet for oppdatering av rutetider og forsinkelser,
- integrasjon mot eksterne datakilder

Dette laget eksponerer et REST API som frontend kommuniserer med.

3. Datakilder

Løsningen henter data fra flere kilder:

- **Intern database (PostgreSQL)** – inneholder brukere, ruter, holdeplasser, avganger, billetter og forsinkelser.
- **Eksterne tjenester** – eksempelvis nasjonale transportdata for sanntidsinformasjon.
- **Administratorinput** – manuelle avviksoppdateringer registrert fra backend eller adminpanelet.

4. Administrasjonsgrensesnitt

Et eget webbasert panel for ansatte og administratorer hvor de kan:

- oppdatere ruteinformasjon,
- registrere og endre forsinkelser,
- administrere brukere og roller,
- inspisere statistikk og systemhelse.

Dette bidrar til at informasjonen som vises i appen alltid er oppdatert.

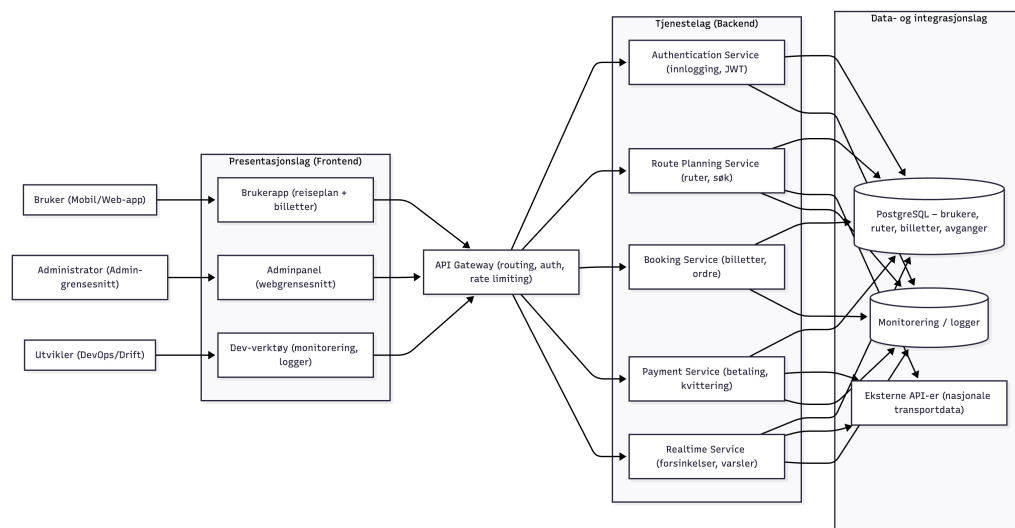
3.4 Konseptuell arkitekturmodell

Figur 3.1 viser en konseptuell oversikt over løsningens arkitektur. Modellen illustrerer hvordan frontend, backend, datakilder og eksterne systemer henger sammen.

Arkitekturen er modulært oppbygd for å gjøre systemet skalerbart og enkelt å videreutvikle. For eksempel kan funksjoner som push-varsler, flerspråklig støtte eller avansert sanntidsanalyse legges til senere uten å påvirke eksisterende moduler i stor grad.

3.5 Oppsummering

Den foreslåtte løsningen representerer en helhetlig og moderne kollektivtransportapplikasjon som forbedrer dagens brukeropplevelse betydelig. Den kombinerer et brukervennlig grensesnitt, pålitelig backend og fleksible datakilder for å møte de viktigste behovene identifisert gjennom brukeranalysen. Løsningen er planlagt slik at den kan videreutvikles gradvis i takt med nye krav, teknologier og brukerbehov.



Figur 3.1: Konseptuell arkitekturmodell for den foreslåtte løsningen.

Kapittel 4

Kravspesifikasjon

4.1 Funksjonelle krav

Funksjonelle krav beskriver hva systemet skal gjøre, og definerer den faktiske funksjonaliteten som brukerne har behov for. Disse kravene er direkte utledet fra personas, scenarier og brukerhistorier, og danner grunnlaget for både utvikling og testing av systemet.

4.1.1 MVP Funksjonelle krav

F01 Brukerautentisering og roller:

Systemet skal ha et sikkert innloggingssystem med fire brukertyper: vanlig bruker, ansatt, administrator og utvikler. Hver rolle skal ha tilgang til spesifikke funksjoner basert på sitt tilgangsnivå. Implementasjonen kan baseres på en etablert autentiseringsløsning, men det konkrete teknologivalget beskrives i arkitekturkapittelet.

F02 Ruteplanlegging:

Systemet skal vise søkbare reiseruter fra valgt start- til sluttdestinasjon, inkludert estimert reisetid og eventuelle bytter. Brukeren skal kunne søke på både holdeplassnavn og adresser.

F03 Oversikt over avganger:

Systemet skal vise en enkel oversikt over bussavganger med tidspunkt, holdeplasser og rutenummer. Informasjonen skal være sortert kronologisk og kunne filtreres.

F04 Digital billettering:

Systemet skal tilby kjøp, validering og fremvisning av bussbilletter gjennom QR-kode. Billetter skal kunne kjøpes i appen og valideres av sjåfør ved skanning.

F05 Grunnleggende sanntidsinformasjon:

Systemet skal vise forsinkelser og avvik fra rutetabellen. For MVP implementeres dette som dummy-data, med mulighet for manuell administrativ oppdatering.

F06 Administrasjonsgrensesnitt:

Systemet skal ha et webbasert administrasjonsgrensesnitt der ansatte og administratorer kan oppdatere rutetider, registrere forsinkelser og administrere brukerkontoer.

F07 CI/CD-pipeline:

Systemet skal benytte en automatisert arbeidsflyt (CI/CD) der GitHub-push validerer kode, kjører tester i en testcontainer og automatisk deployer til produksjonsmiljø dersom alle tester består. Dette er en kontinuerlig prosess uten slutt dato.

4.1.2 Utvidede funksjonelle krav

Dette er funksjoner som ikke inngår i MVP, men som vurderes å implementeres dersom tid og kapasitet tillater det.

F08 Flerspråklig støtte:

Systemet skal støtte norsk som hovedspråk og engelsk som sekundært språk. Dette adresserer behovet til brukere med begrensede norskkunnskaper.

F09 Push-varsler:

Systemet skal kunne sende push-varsler om forsinkelser, ruteendringer og viktige oppdateringer.

F10 Offline-funksjonalitet:

Systemet skal cache grunnleggende rutedata og kjøpte billetter for bruk uten internettilkobling.

F11 Avansert sanntidsdata:

Systemet skal hente sanntidsdata fra eksterne API-er (f.eks. nasjonale transportdata) og kontinuerlig oppdatere ruteinformasjon.

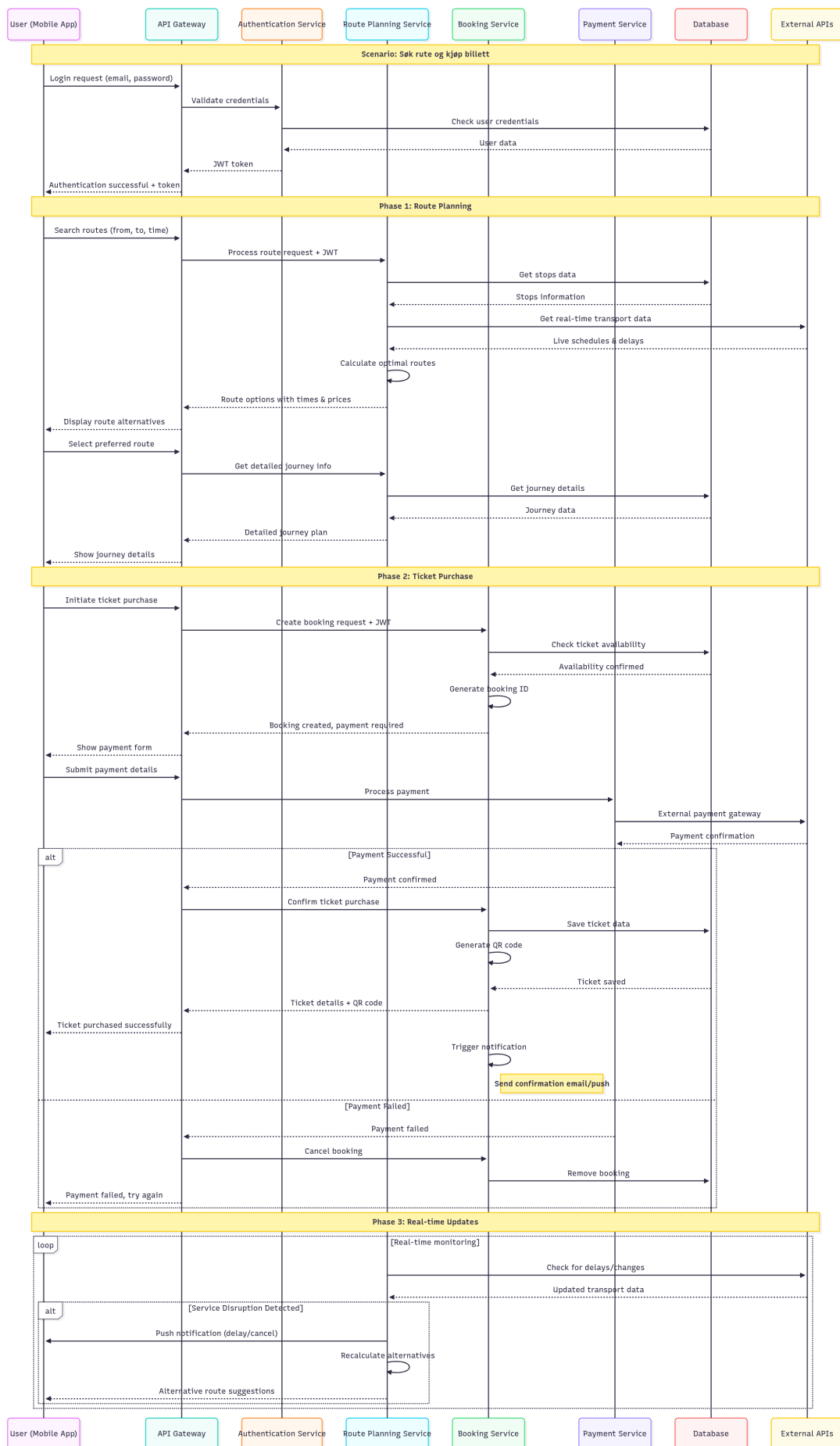
F12 Reiseplanlegger med preferanser:

Systemet skal gi brukeren mulighet til å velge preferanser som raskeste reise eller færrest bytter.

Vi valgte disse kravene fordi de støtter konkrete behov fra våre personas: F05 møter Philips behov for sanntidsinformasjon, F04 støtter Emmas behov for digital billettering, F08 hjelper Adam som har språkutfordringer, og F06 støtter Lars i hans rolle som administrator. Denne koblingen sikrer at kravene er både nødvendige og brukerorienterte.

4.1.3 Sekvensdiagram for kjerneprosess

For å tydeliggjøre hvordan de funksjonelle kravene realiseres, viser figur 4.1 en kjerneprosess der en bruker logger inn, søker etter en rute, kjøper billett og mottar en QR-kode. Prosessen involverer mobilapp, API-gateway, autentiseringstjeneste, ruteplanlegging, booking, betalingstjeneste, database og eksterne API-er.



Figur 4.1: Sekvensdiagram for prosessen søk av rute og billettkjøp

Som vist i diagrammet:

- Autentisering (F01) utføres først, og brukeren mottar en JWT-token.
- Ruteplanlegging (F02) utføres basert på registrerte stopp og sanntidsdata.
- Billettering (F04) genererer booking, betaling og QR-kode.
- Sanntidsoppdateringer (F05) håndteres ved å overvåke forsinkelser og generere varsler (eventuelt i en fremtidig versjon).

4.2 Ikke-funksjonelle krav

Ikke-funksjonelle krav beskriver hvordan systemet skal være, og omhandler kvaliteter som ytelse, sikkerhet og brukervennlighet. Disse kravene er avgjørende for et produksjonsklart system, men kun deler av dem er nødvendige for MVP.

4.2.1 Ytelse og skalerbarhet

- Systemet skal håndtere minimum 1000 samtidige brukere uten merkbar ventetid.
- API-endepunkter skal ha maksimal responstid på 2 sekunder under normal belastning.
- Appen skal starte innen 5 sekunder på moderne enheter (siste 5 år).
- Systemet skal ha minst 98% oppetid i normal drift.

4.2.2 Sikkerhet og personvern

- Systemet skal være i samsvar med GDPR, inkludert brukersamtykke og rett til sletting.
- All kommunikasjon skal krypteres med TLS 1.3.
- Passord skal hashes med bcrypt med salt.
- Systemet skal bruke JWT-tokens med automatisk utløp.
- Mislykkede innloggingsforsøk skal logges og gjentatte forsøk midlertidig blokkeres.

4.2.3 Brukervennlighet og tilgjengelighet

- Grensesnittet skal være intuitivt for førstegangsbrukere.
- Systemet skal oppfylle WCAG 2.1 AA for universell utforming.
- Appen skal fungere på skjermstørrelser fra 4" til 13".
- Systemet skal støtte skjermlesere og høy kontrast.

4.2.4 Plattform og kompatibilitet

- Appen skal utvikles i React Native for både Android og iOS.
- En webversjon skal tilgjengeliggjøres for administrasjonsfunksjoner.
- Appen skal oppfylle kravene for publisering i App Store og Google Play.

4.2.5 Utviklings- og driftskrav

- Backend skal ha minimum 80% testdekning, frontend minimum 70%.
- CI/CD skal kjøre automatiserte tester for hver push.
- Spring Boot Actuator skal benyttes for health checks.
- Kodebasen skal følge etablerte kodestandarder og ha dokumentasjon.

4.3 MVP Tekniske spesifikasjoner

4.3.1 Minimumskrav for MVP

Teknisk arkitektur

Backend (Java Spring Boot)

- REST API med minimum seks endepunkter: autentisering, ruteplanlegging, billetter, administrasjon.
- PostgreSQL-database med tabeller for brukere, ruter, billetter, avganger og forsinkelser.
- Autentisering via Spring Security og JWT.
- Grunnleggende feilhåndtering og logging.
- Distribuert som Docker-container.

Frontend (React Native)

- Cross-platform-app for Android (primært) og iOS.
- Fire hovedskjermer: innlogging, ruteplanlegging, billettoversikt og adminpanel.
- Generering og visning av QR-kode.
- Navigasjon via React Navigation.

Database (PostgreSQL)

- Minimum fem relaterte tabeller.
- Dummy-data for testing og demonstrasjon.
- Enkel backupstrategi.

Validering og health checks

Automatiserte tester

- Enhetstester for kritisk forretningslogikk.
- Integrasjonstester mot database.
- API-tester for hovedfunksjoner.
- UI-tester for kritisk brukerflyt.

Health checks

- `/actuator/health` — systemhelse
- `/actuator/health/db` — databasetilkobling
- Egne indikatorer ved behov

4.4 Kravsoversikt med estimer

4.4.1 Estimering og story points

Vi vurderte flere estimeringsteknikker før vi valgte **Planning Poker** som metode for å estimere kompleksitet og innsats. Planning Poker er valgt fordi det kombinerer individuell vurdering med gruppediskusjon, og fører til felles forståelse av arbeidsmengde.

Teamet bruker en modifisert Fibonacci-skala (1, 2, 3, 5, 8, 13, 21), der høyere tall indikerer større kompleksitet, risiko eller omfang. Story points måler innsats og kompleksitet relativt mellom oppgaver, og brukes senere til sprintplanlegging via teamets velocity.

Total kompleksitet for MVP: 61 story points.

Estimert tidsramme: 9 uker utviklingstid.

ID	Krav	Story Points	Prioritet	Forretningsnytte	MVP
F01	Brukerautentisering og roller	2	Høy	Høy	ja
F02	Ruteplanlegging	7	Høy	Høy	ja
F03	Oversikt over avganger	7	Høy	Høy	ja
F04	Digital billettering	3	Høy	Høy	ja
F05	Grunnleggende sanntidsinfo	5	Høy	Middels	ja
F06	AdministrasjonsUI	3	Middels	Middels	ja
F07	CI/CD Pipeline	Kontinuerlig	Høy	Lav	ja
F08	Flerspråklig støtte	3	Middels	Høy	
F09	Push-varsler	3	Middels	Høy	
F10	Offline-funksjonalitet	4	Lav	Middels	
F11	Avansert sanntidsdata	12	Lav	Høy	
F12	Reiseplanlegger preferanser	12	Lav	Lav	

Tabell 4.1: Oversikt over funksjonelle krav med estimater og MVP-prioritering

4.5 Akseptansekriterier for MVP

4.5.1 Tekniske kriterier

- Systemet kjører stabilt i minst 24 timer uten restart.
- Alle health checks rapporterer grønn status.
- CI/CD-pipelen kjører uten feil etter push til main-branch.
- Appen kan installeres og kjøres på minst to ulike Android-enheter.

4.5.2 Funksjonelle kriterier

- Brukeren kan planlegge reise og kjøpe billett (F02 + F04) på under ett minutt.
- Administrator kan oppdatere rutetider (F06), og endringen gjenspeiles i appen.
- QR-kode generert i appen kan skannes av en standard QR-skanner.
- Billetter er tilgjengelige i offline-modus (F10) hvis funksjonen er aktivert.

4.5.3 Brukbarhetskriterier

- En bruker med lav digital kompetanse (Berit, 72) kan navigere til billettkjøp uten hjelp.
- En ung bruker (Philip, 10) forstår hvordan forsinkelser vises.
- Appen fungerer i både portrett- og landskapsmodus.

Kapittel 5

Arbeidsprosess og organisering

Dette kapitlet beskriver hvordan vi organiserte arbeidet i prosjektet, hvordan vi fordelte oppgaver, og hvilke metoder og verktøy vi benyttet som team. Som et team på fem studenter med ulik erfaring var det viktig å etablere en struktur som både støttet læring og sikret jevn fremdrift mot MVP-målet.

5.1 Arbeidsmetodikk

5.1.1 Utvikling med Scrum

Vi valgte å følge Scrum-rammeverket, tilpasset et studentprosjekt. Arbeidet ble organisert i sprinter med følgende overordnede struktur:

Sprintlengde: 2 uker (totalt 7 sprinter)

- Sprint 1–2: Prosjektoppsett, arkitektur og grunnleggende autentisering.
- Sprint 3–4: Kjernefunksjonalitet (ruteplanlegging og billettbestilling).
- Sprint 5–6: Integrasjon, testing og administrasjons-UI.
- Sprint 7: Finpussing, dokumentasjon og deployment.

Roller og ansvar:

- **Philip:** Scrum Master og teknisk leder, med ansvar for arkitektur og koordinering av tekniske beslutninger.
- **Erica:** Product Owner, ansvarlig for kravprioritering, brukerhistorier og dokumentasjon.
- **Simon:** Frontend-spesialist, leder for utvikling av React Native-applikasjonen.
- **Jimmy:** Backend-spesialist, med hovedansvar for Java Spring Boot-API og databaseoperasjoner.

- **Mohamed:** DevOps- og testansvarlig, med fokus på CI/CD-pipeline, kvalitetssikring og automatiserte tester.

5.1.2 Agile seremonier

Daily standups (inntil 3 ganger per uke)

Standups ble gjennomført digitalt via Discord. Hver deltaker svarte kort på:

- Hva gjorde jeg siden sist?
- Hva skal jeg gjøre til neste gang?
- Er det noen hindringer?
- Trenger jeg hjelp fra noen?

Sprint planning

I starten av hver sprint gjennomførte vi en planleggingsøkt på omtrent to timer der teamet:

- Gikk gjennom product backlog sammen med Product Owner.
- Estimerte oppgaver ved hjelp av Planning Poker.
- Valgte oppgaver til sprint backlog basert på kapasitet og prioritet.
- Definerte tydelige akseptansekriterier for hver oppgave.

Sprint review og retrospektiv

Ved slutten av hver sprint gjennomførte vi en kombinert review og retrospektiv:

- **Review:** Vi demonstrerte ferdig funksjonalitet, fikk tilbakemeldinger og justerte product backlog ved behov.
- **Retrospektiv:** Vi reflekterte over hva som fungerte bra, hva som kunne forbedres, og formulerte konkrete tiltak for neste sprint.

5.2 Verktøy og teknisk infrastruktur

5.2.1 Versjonskontroll og samarbeid

Git og GitHub

Vi brukte Git som versjonskontrollsystem og GitHub som plattform for samarbeid.

Repository-informasjon:

- **Repository-URL:** <https://github.com/craftpag/crispy-bassoon-ostfold>
- **Tilgangsstyring:** Repositoryet er tilgjengelig for alle gruppelemmer (Jimmy, Erica, Philip, Simon, Mohamed), emneansvarlig og veileder.
- **Struktur:** Monorepo med separate kataloger for **backend**, **frontend** og **docs**.

Branching-strategi (GitFlow-inspirert):

- **main:** Produksjonsbranch som kun inneholder ferdig testet funksjonalitet. Det merges kun inn fra **develop** ved slutten av en sprint.
- **develop:** Integrasjonsbranch hvor ferdige features samles og testes før produksjonssetting.
- **feature-branches:** Opprettes fra **develop** for hver MVP-funksjonalitet, for eksempel:
 - **feature/user-authentication** (F01)
 - **feature/route-planning** (F02)
 - **feature/departure-overview** (F03)
 - **feature/digital-ticketing** (F04)
 - **feature/realtime-info** (F05)
 - **feature/admin-interface** (F06)
 - **feature/ci-cd-pipeline** (F07)

Samarbeidsrutiner:

- **Code review:** Alle pull requests skulle godkjennes av minst én annen utvikler.
- **Commit-konvensjoner:** Vi fulgte *Conventional Commits* for å legge til rette for automatisk changelog-generering.
- **Konfliktløsning:** Feature-branches ble jevnlig oppdatert fra **develop** for å redusere merge-konflikter.

GitHub Actions og CI/CD

- Automatisk bygging og testing ved pull requests.
- Kvalitetssjekk av kode (linting og tester).
- Automatisk deployment til staging-miljø ved merge til **develop**.
- Produksjonsdeployment krever manuell godkjenning.
- Pipelineen er direkte koblet til MVP-kravet F07 (CI/CD-pipeline).

Begrunnelse for GitHub:

Vi valgte GitHub fremfor alternativer som Azure Repos på grunn av god integrasjon med open source-verktøy, støtte for private repositories og at alle i teamet hadde erfaring med GitHub fra tidligere prosjekter.

5.2.2 Prosjektstyring og kommunikasjon

Jira for oppgavehåndtering

- **Epics:** Organisert etter hovedfunksjoner (autentisering, ruteplanlegging, billettering, administrasjon osv.).
- **Story mapping:** Brukt til å visualisere brukerreisene og prioritere funksjonalitet.
- **Burndown charts:** Benyttet til å følge fremdrift per sprint.
- **Velocity tracking:** Registrering av teamets leveringskapasitet over tid.

Kommunikasjonsverktøy

- **Discord:** Daglig kommunikasjon, uformelle møter, pair programming og voice calls.
- **Microsoft Teams:** Formelle møter, veiledning og skjermdeling.
- **Notion:** Felles kunnskapsbase, møtereferater og dokumentstruktur.
- **Figma:** Prototyping og design-samarbeid.

5.2.3 Utviklingsverktøy

Backend-utvikling

- **IntelliJ IDEA:** Felles IDE for Java-utvikling.
- **Postman:** API-testing og dokumentasjon.
- **pgAdmin:** Databaseadministrasjon og SQL-spørringer.
- **Docker:** Lokalt utviklingsmiljø og deployment-oppsett.
- **Backend-host:** `itstud.hiof.no/~xx`

Frontend-utvikling

- **VS Code / Cursor:** Utvikling av React Native-kode.
- **React Native Debugger:** Feilsøking, state-inspeksjon og ytelsesanalyse.
- **Android Studio / Xcode:** Emulering og testing på fysiske og virtuelle enheter.

5.3 Oppgavefordeling og samarbeid

5.3.1 Kompetansebasert fordeling

Philip (teknisk leder)

- Overordnet ansvar for backend-arkitektur og Spring Boot-oppsett.
- Database-design og optimalisering av PostgreSQL.
- Tekniske beslutninger og støtte til øvrige utviklere.
- Oppsett av arbeidsflyt og retningslinjer.

Simon (frontend-spesialist)

- Struktur og navigasjon i React Native-applikasjonen.
- Implementasjon av UI/UX basert på Figma-design.
- State management og eventuell offline-funksjonalitet.
- Forberedelse til publisering i app-butikker.

Jimmy (backend-spesialist)

- Utvikling og dokumentasjon av REST API-er.
- Implementasjon av sikkerhet og autentisering.
- Databaseoperasjoner og optimalisering.

Mohamed (QA- og DevOps-ansvarlig)

- Oppsett og vedlikehold av CI/CD-pipeline.
- Automatiserte tester (unit-, integrasjons- og E2E-tester).
- Ytelsestesting og monitorering.
- Deployment og håndtering av infrastruktur.

Erica (Product Owner)

- Kravanalyse, utforming av brukerhistorier og oppfølging av product backlog.
- Prioritering og koordinering mot MVP-mål.
- Akseptansetesting og brukervalidering.
- Hovedansvar for rapport og prosjektdokumentasjon.

5.3.2 Pair programming og kunnskapsdeling

Planlagt pair programming

- Komplekse oppgaver ble i størst mulig grad løst i par for å sikre kvalitet og kunnskapsdeling.
- Teamet diskuterte jevnlig hvem som tok hvilke oppgaver, slik at alle fikk arbeide både innenfor og utenfor egen komfortsone.

Kapittel 6

Sprint 1 – Planlegging og gjennomføring

6.1 Sprintmål

Hovedmålet med Sprint 1 var å etablere et solid fundament for prosjektet ved å utvikle et fungerende system for brukerautentisering. Dette innebar å sette opp en database med tabeller for brukere og roller, samt håndtering av brukernavn, passord og tilgangsnivå. I tillegg ønsket vi å implementere funksjonalitet for innlogging og registrering i backend, slik at brukere kunne opprette konto og logge inn på en sikker måte.

For at systemet skulle være tilgjengelig for sluttbruker, skulle vi også utvikle en enkel frontend med skjemaer for innlogging og registrering, koblet mot backend via REST API. Dette skulle sikre kommunikasjon mellom brukergrensesnitt og database. Videre skulle vi sette opp en CI/CD-pipeline i GitHub for å automatisere bygging og testing, slik at vi tidlig la til rette for effektiv utvikling videre i prosjektet.

Sprintmålet henger tett sammen med prosjektets overordnede mål, ettersom autentisering er kjernen i et sikkert og brukervennlig system. Uten dette ville det ikke vært mulig å bygge videre på funksjonalitet som krever innlogging og personlige preferanser. Når Sprint 1 var fullført, hadde vi dermed lagt fundamentet som resten av systemet kunne bygges videre på.

Sprintmålet konkretiseres slik:

- Oppsett av database med brukertabeller, roller og passordhåndtering.
- Backend-funksjonalitet for innlogging og registrering (Java Spring Boot).
- Frontend-skjemaer for innlogging og registrering, koblet mot backend via REST API.
- CI/CD-pipeline i GitHub for bygging og testing.

All kildekode for prosjektet er tilgjengelig på: github.com/craftpag/crispy-bassoon-ostfold.

6.2 Sprint backlog

Oppgave	Estimat (timer)	Ansvarlig
Databaseoppsett (brukertabeller, roller, passordhashing)	8	Erica
Login/registrering backend (Java)	12	Jimmy
Frontend login-/registreringsdesign	10	Simen
REST API-kobling (frontend ↔ backend)	10	Philip
CI/CD-pipeline (GitHub Actions, test-script)	6	Mohamed

Tabell 6.1: Sprint backlog for Sprint 1

Order	ID	Title	Assigned To	State	Tags
1	19	Deployment on dev server	Philip André G...	To Do	
2	18	CI/CD Pipeline Validation of code	Mohamed Abd...	To Do	
3	17	RESTapi CORS setup	Philip André G...	To Do	
4	16	Frontend for login/registrering	Simen Kingsrød	To Do	
5	15	Login/registreringssystem in java	Jimmy Bui	To Do	
6	14	Database Setup	Erica Bich Che	To Do	

Figur 6.1: Backlog for Sprint 1

Totalt estimert arbeid var 46 timer, noe som var vurdert som realistisk innenfor sprintens tidsramme på to uker.

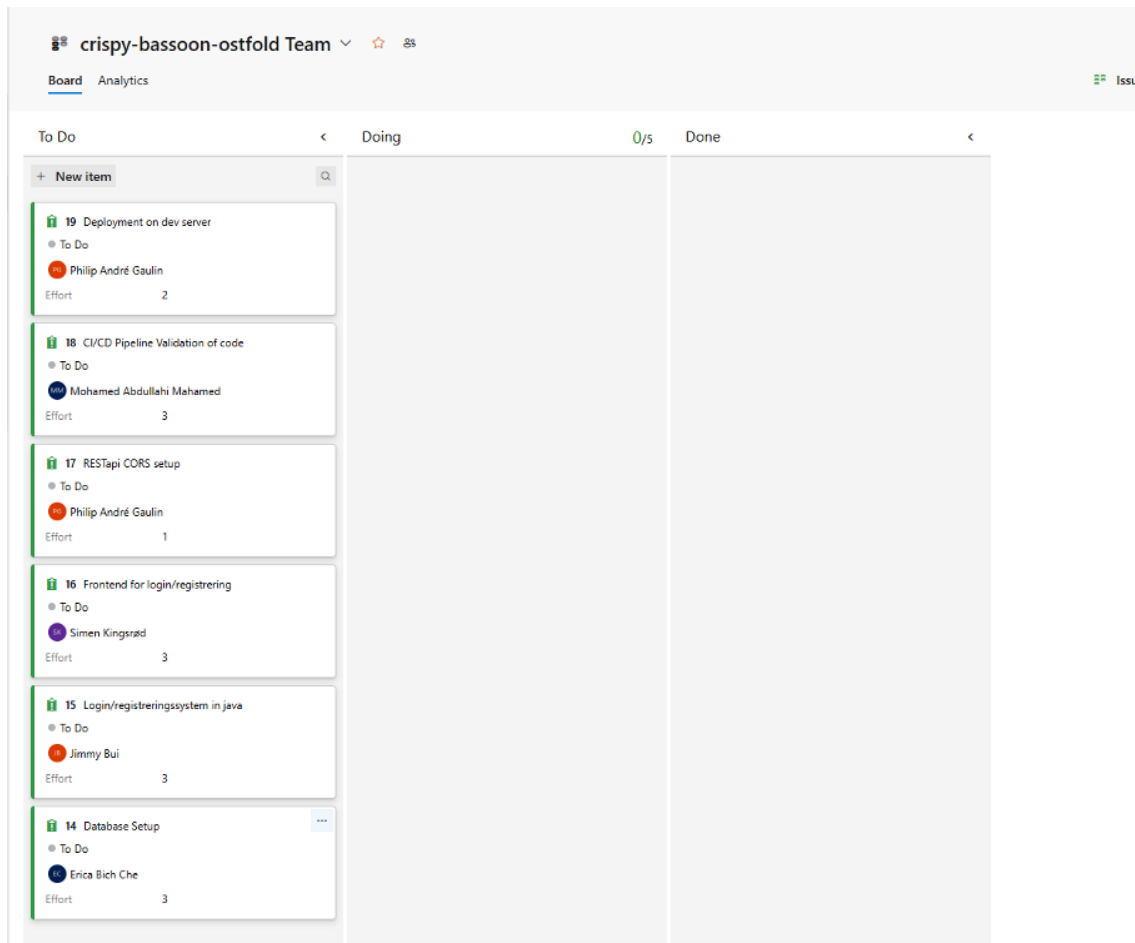
6.3 Kanban-board

Vi organiserte oppgavene i et kanban-board med fire kolonner:

- **To Do** – oppgaver valgt for sprinten.
- **In Progress** – pågående arbeid.
- **Testing/Review** – klar for testing eller code review.
- **Done** – ferdige oppgaver.

Milepæler:

- Sluttet av uke 1: Database og backend for login/registrering ferdigstilles.
- Sluttet av uke 2: Frontend, API-kobling og CI/CD-pipeline fullføres og integreres.



Figur 6.2: Kanban-board for Sprint 1

6.4 Ansvarsfordeling

- Erica: Databaseoppsett.
- Jimmy: Login-/registreringslogikk i backend.
- Simen: Frontend-design for innlogging og registrering.
- Philip: REST API-kobling og oppsett av produksjonsserver.
- Mohamed: CI/CD-pipeline.
- Alle: Testing og dokumentasjon.

6.5 Sprint 1 – Retrospektiv

Sprint 1 ble gjennomført i perioden 22.09.2025–06.10.2025, med hovedmål om å etablere et sikkert og fungerende autentiseringssystem. Alle planlagte oppgaver ble fullført innenfor sprintens tidsramme, og resultatet ble et stabilt fundament for videre utvikling.

Hva fungerte bra

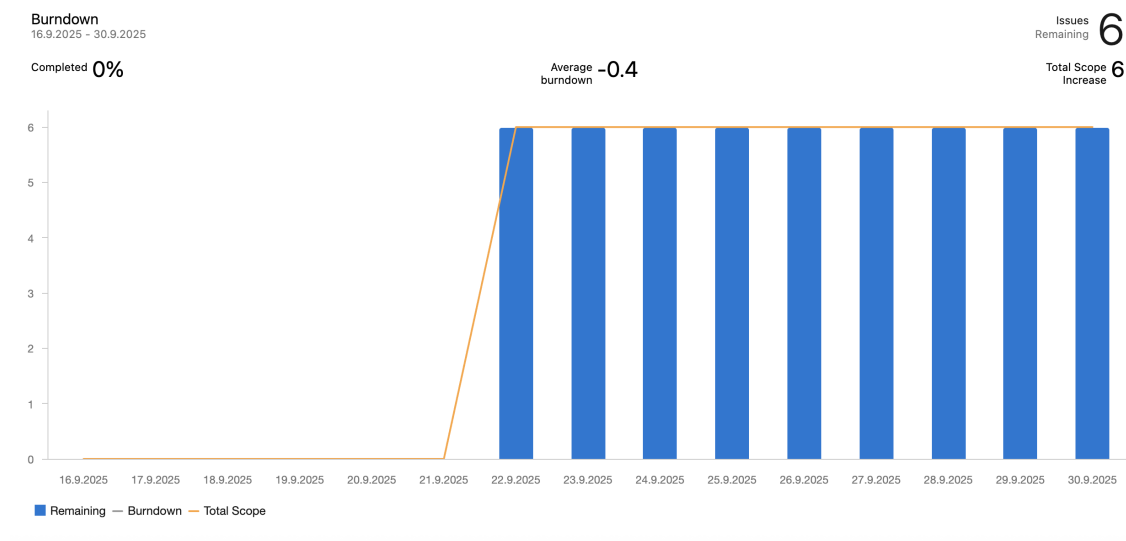
Planleggingen av sprinten fungerte godt. Vi fikk bedre kontroll på hvilke oppgaver som skulle være gjennomført innen en gitt tidsfrist, og hver deltaker fikk tydelig definerte ansvarsområder. Dette gjorde det enklere å fokusere på én konkret oppgave av gangen. Felles planlegging av sprinten bidro også til en felles forståelse av mål og leveranser.

Hva kunne vært bedre

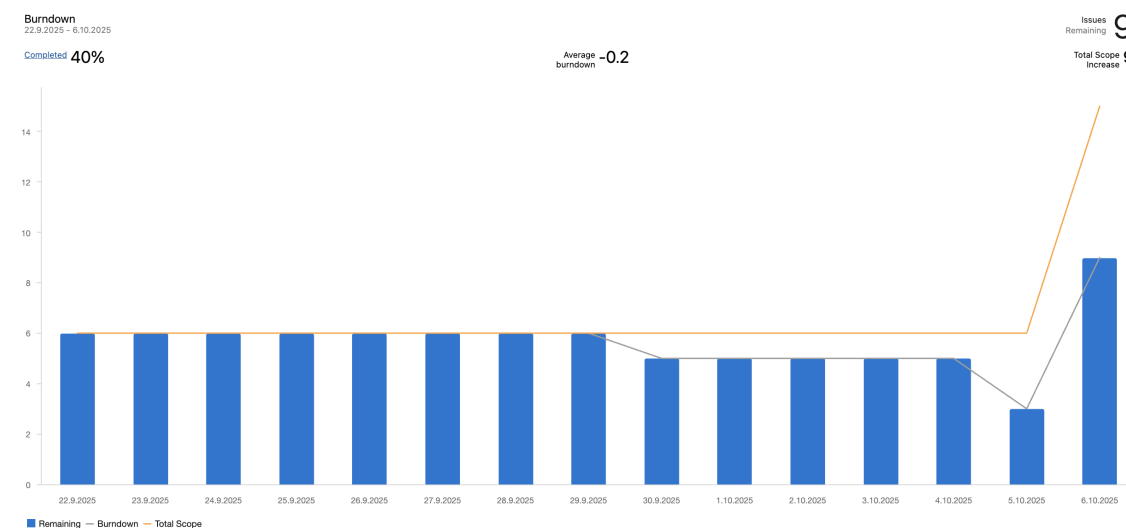
Utførelsen av oppgavene innen fristen kunne likevel vært jevnere fordelt. Vi erfarte at kommunikasjonen ble mer utfordrende når vi ikke jobbet fysisk sammen, og enkelte avklaringer tok lengre tid enn planlagt. I tillegg viste det seg at noen oppgaver var mer komplekse enn først antatt, spesielt med tanke på gruppens erfaring med Spring Security og CI/CD.

Hva tar vi med videre

For Sprint 2 ønsker vi å forbedre kommunikasjonen ytterligere, blant annet ved å ha hyppigere statusoppdateringer digitalt. Vi vil også være mer bevisste på å følge opp hverandres fremdrift underveis i sprinten, slik at utfordringer fanges opp tidlig og kan løses raskere.



Figur 6.3: Burndown chart for Sprint 1

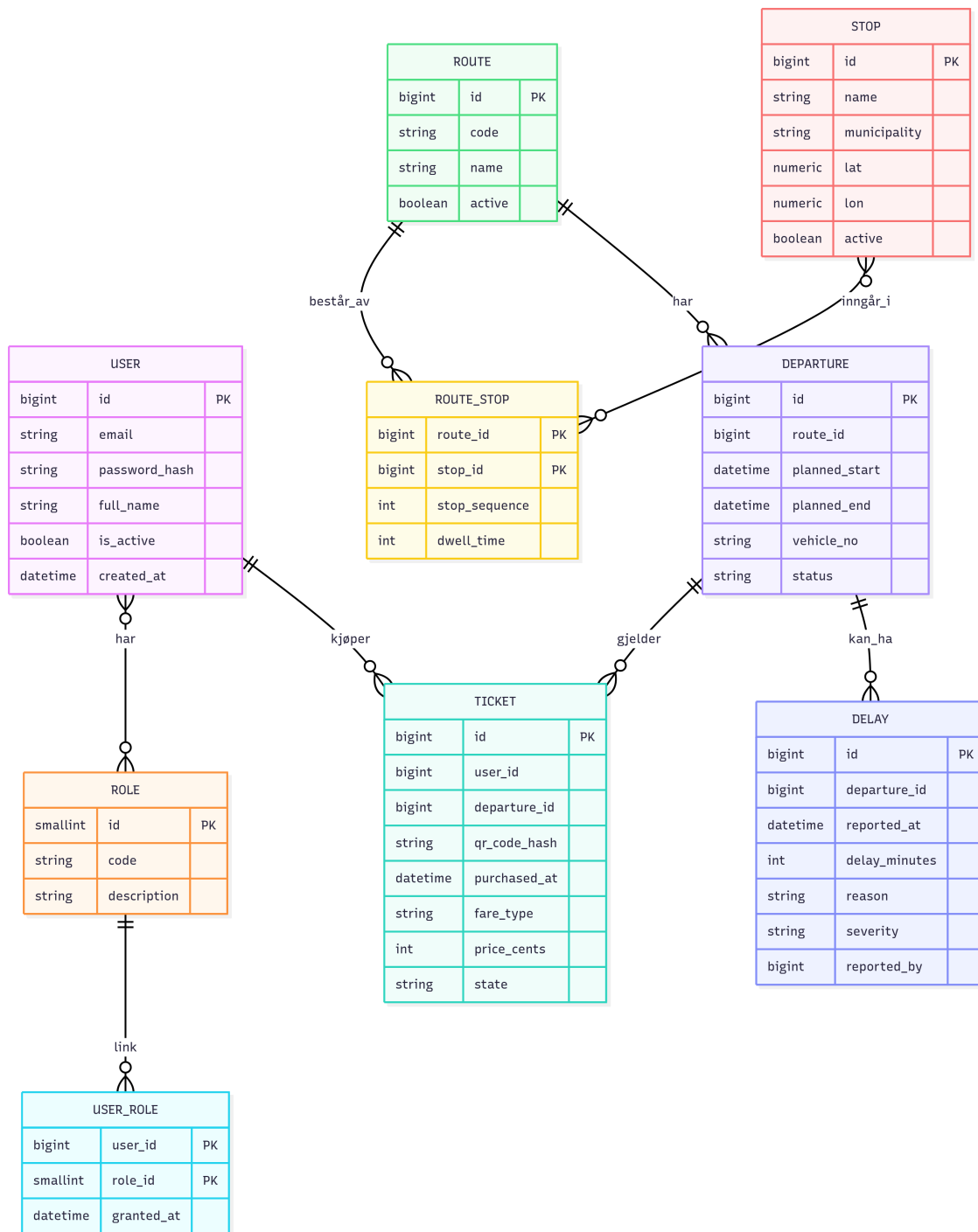


Figur 6.4: Burndown chart på slutten av Sprint 1

Figur 6.3 og figur 6.4 viser utviklingen i sprinten. Kurven er forholdsvis flat i starten, før den faller brattere mot slutten. Dette skyldes at teamet bevisst fordelte arbeidet slik at hver deltaker hadde én større oppgave å fokusere på, fremfor mange små. Denne arbeidsformen gjorde at alle kunne jobbe mer selvstendig og fordype seg i sitt ansvarsområde.

6.6 Datamodell (ER-diagram)

Figur 6.5 viser ER-diagrammet for bussappens MVP-database. Diagrammet illustrerer hvordan tabellene henger sammen, og hvilke attributter som er nødvendige for å støtte kjernefunksjonaliteten (ruteplanlegging, billettering, autentisering og sanntidsinformasjon).



Figur 6.5: ER-diagram for bussappens MVP-database

Tabellbeskrivelser

- **USER**: Lagrer brukerinformasjon med unik e-postadresse, passordhash, fullt navn og aktiv-status.

- **ROLE** og **USER_ROLE**: Håndterer tilgangsstyring. En bruker kan ha flere roller (f.eks. ADMIN, EMP, USER), og mange-til-mange-forholdet løses med koblingstabellen **USER_ROLE**.
- **ROUTE**: Representerer en busslinje (f.eks. rute 633), med kode, navn og aktiv-status.
- **STOP** og **ROUTE_STOP**: Beskriver holdeplasser og deres plassering i en rute. Mange-til-mange-forholdet løses med **ROUTE_STOP**, som også lagrer rekkefølgen (**stop_sequence**).
- **DEPARTURE**: En konkret avgang for en bestemt rute på et gitt tidspunkt, med planlagt start- og sluttid, samt status (PLANNED, CANCELLED, DONE osv.).
- **DELAY**: Registrerer forsinkelser knyttet til avganger. Lagrer antall minutter, tidspunkt, alvorlighetsgrad og hvem som har rapportert avviket.
- **TICKET**: Digitale billetter, knyttet til både bruker og avgang. Lagrer kjøpstidspunkt, billetttype (ADULT, STUDENT, SENIOR), pris og status (NEW, USED, REFUNDED). Billetten identifiseres gjennom en unik QR-kode.

Relasjoner

- En **User** kan ha flere **Tickets**, og kan tildeles en eller flere **Roles**.
- En **Route** består av flere **Stops**, og hvert stopp har en definert rekkefølge gjennom **Route_Stop**.
- En **Route** kan ha mange **Departures**, og hver avgang kan ha null eller flere **Delays**.
- En **Ticket** kobles til både **User** og **Departure**, noe som gjør det mulig å validere hvem som eier billetten og hvilken avgang den gjelder.

6.6.1 Databaseoppsett i PostgreSQL

For å realisere datamodellen (figur 6.5) satte vi opp databasen i PostgreSQL 16 ved hjelp av pgAdmin 4. Vi definerte roller, skema og tabeller i tråd med ER-diagrammet, og la inn seed-data for å kunne teste løsningen tidlig.

Oppsett

Oppsettet ble gjort i tre steg:

1. Opprettelse av rollene **app_admin** (full kontroll) og **app_user** (applikasjonsbruker).
2. Opprettelse av databasen **bussapp_db** og skjemaet **app**.
3. Kjøring av et **init.sql**-skript som opprettet tabeller, ENUM-typer, indekser og la inn seed-data.

Eksempel på SQL

Under er et utdrag fra SQL-skriptet som opprettet **user**-tabellen. Alle tabeller følger tilsvarende struktur med primærnøkler, fremmednøkler og constraints.

Kode 6.1: Opprettelse av brukertabellen

```
CREATE TABLE app."user" (
  id BIGSERIAL PRIMARY KEY,
  email CITEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  full_name TEXT NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Roller og rettigheter

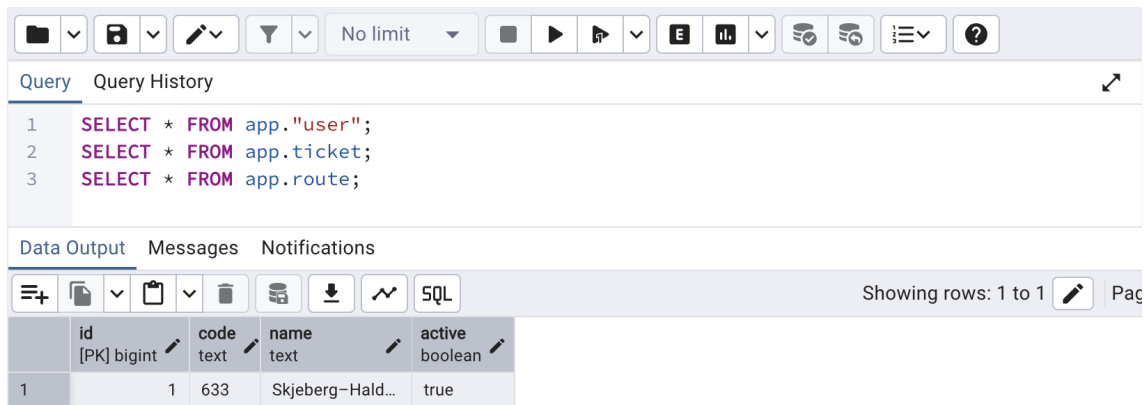
Vi satte opp roller og rettigheter for å skille mellom administrasjon og applikasjonsbruk. Dette bidrar til bedre sikkerhet og kontroll over databasen.

Rolle	Rettigheter	Beskrivelse
app_admin	CREATE, ALTER, DROP, SELECT, INSERT, UPDATE, DELETE	Eierskap til skjema app . Full kontroll, brukes kun til administrasjon og vedlikehold.
app_user	SELECT, INSERT, UPDATE, DELETE	Brukes av applikasjonen. Kan lese og endre data i tabellene, men kan ikke endre struktur eller skjema.
postgres	Alle	Kun brukt ved initielt oppsett og installasjon av utvidelser.

Tabell 6.2: Oversikt over roller og rettigheter i databasen

Verifisering i pgAdmin

For å verifisere at oppsettet fungerte, kjørte vi spørringer direkte i Query Tool i pgAdmin. Figur 6.6 viser seed-data fra tabellen **ticket**, generert automatisk i init-skriptet.



Figur 6.6: Seed-data verifisert i pgAdmin

Med dette oppsettet har vi en database som støtter autentisering, billettering, ruteinformasjon og sanntidsdata. Dette danner fundamentet for integrasjonen med backend (Spring Boot) og frontend (React Native).

6.7 Oppsett av produksjonsserver

Produksjonsserveren for prosjektet er satt opp med en automatisert deployment-pipeline som håndterer hele flyten fra kodeendring til kjørende applikasjon.

6.7.1 Deployment-flyt

Når det pushes en oppdatering til **main**-branchen i repositoryet, starter en automatisert prosess. Koden går først gjennom en pipeline som validerer endringene. Etter godkjenning blir den nye koden automatisk hentet ned til produksjonsserveren. Et shell-script kjører deretter nødvendige kommandoer for å oppdatere både backend og frontend, før applikasjonen startes på nytt med den oppdaterte koden.

6.7.2 Shell-script-funksjonalitet

Deploy-scriptet håndterer en rekke oppgaver automatisk. Det stopper først kjørende tjenester, henter siste versjon fra Git, og installerer eventuelle nye avhengigheter. Deretter bygges frontend-ressursene før backend og frontend startes på nytt. Til slutt verifiserer scriptet at alle tjenester kjører som forventet.

6.7.3 Forutsetninger for vellykket deployment

Automatiseringen fungerer problemfritt så lenge oppstartskommandoene for applikasjonen forblir uendret og filstrukturen er konsistent. Større endringer i build-prosessen kan kreve justeringer i deploy-scriptet.

6.7.4 Teknisk gjenbruk

Shell-scriptet er gjenbrukt fra et tidligere prosjekt. Dette reduserte tidsbruk og risiko, ettersom scriptet allerede var testet og verifisert i praksis. Ved å bygge videre på eksisterende infrastruktur kunne vi fokusere mer på funksjonalitet enn på driftsoppsett.

6.7.5 Fordeler med oppsettet

Oppsettet gir minimal nedetid under oppdateringer, og en konsistent prosess reduserer risikoen for menneskelige feil sammenlignet med manuelle deployments. Tiden fra push til produksjon er lav, noe som muliggjør rask iterasjon. All deployment-historikk ligger lagret i Git, noe som gir god sporbarhet og mulighet for rollback ved behov.

6.8 Oppsett av login-/registreringssystem

Som del av Sprint 1 utviklet vi et enkelt innloggings- og registreringssystem som del av bussapplikasjonen. Systemet håndterer opprettelse av nye brukere og innlogging av eksisterende brukere via et REST API. Målet var å etablere et fungerende MVP for autentisering, med en struktur som senere kan utvides med mer avansert sikkerhet.

Backend ble utviklet i IntelliJ IDEA med Spring Boot som rammeverk. Vi fulgte en klassisk Controller–Service–Repository-struktur for å holde koden ryddig og lett å videreutvikle.

6.8.1 Utviklingssteg

Arbeidet fulgte disse stegene:

- **Controller-lag:** `AuthController` håndterer HTTP-forespørsler for innlogging og registrering.
- **DTO-er:** `LoginRequest` og `RegisterRequest` brukes for dataoverføring mellom frontend og backend.
- **Modell:** `User`-klassen representerer brukere i systemet.
- **Repository:** `UserRepository` lagrer brukere (i første omgang i et midlertidig lager, senere koblet mot database).

- **Service-lag:** `AuthService` inneholder logikken for registrering og innlogging.

6.8.2 Testing

API-et ble testet ved hjelp av Postman, med **POST**-forespørsler til:

- `/auth/register`
- `/auth/login`

Responsen var som forventet:

- `"User registered!"` ved vellykket registrering.
- `"Username already exists"` dersom brukernavnet allerede var i bruk.
- `Login successful` ved gyldig innlogging.
- `"Invalid credentials"` ved feil brukernavn eller passord.

6.8.3 Resultat

Resultatet ble et enkelt, men fungerende innloggings- og registreringssystem. Løsningen fungerer som et godt grunnlag for videre utvikling. Strukturen gjør det enkelt å senere koble til database, legge til Spring Security og bygge en frontend i React Native.

6.8.4 Konklusjon

Sprint 1 oppnådde målet om å etablere et fungerende autentiseringssystem basert på Java og Spring Boot. Systemet tilbyr grunnleggende autentisering gjennom et API og håndterer både vellykkede og ugyldige forespørsler på en tydelig måte. Gjennom arbeidet fikk teamet bedre forståelse for hvordan backend-systemer bygges opp, og hvordan man kan strukturere kode med controller-, service- og repository-lag. Løsningen danner et solid fundament for videreutvikling, inkludert integrasjon mot database og sikrere passordhåndtering.

Kapittel 7

Sprint 2 – Planlegging og gjennomføring

7.1 Sprintmål

Målet med Sprint 2 var å etablere en fungerende grunnstruktur for systemet, der kjernefunksjoner som sikkerhet, ruteplanlegging og digital billettering er på plass i enkel form. I tillegg skulle databasen fylles med dummy-data, og systemet skulle kunne vise grunnleggende sanntidsinformasjon. Dette gir et utgangspunkt for videre utvikling, testing og iterasjon i neste sprint. Enkelte deler av frontend og design krevde fortsatt justeringer for bedre brukervennlighet og visuell konsistens.

Gjennomførte oppgaver i Sprint 1 (grunnlag for Sprint 2)

- Opprettet og konfigurert REST API.
- Satt opp database og skjema.
- Etablert CI/CD-pipeline i GitHub.
- Implementert innloggings- og registreringssystem.
- Designet frontend for innlogging og registrering.
- Deployert applikasjonen på utviklingsserver.

Hovedmål for Sprint 2

- Etablere brukerroller og forbedret autentisering og sikkerhet.
- Implementere ruteplanlegging.
- Utvikle digital billettering (første versjon).

- Legge inn dummy-data i databasen for ruter og billettsystem.
- Vise grunnleggende sanntidsinformasjon.

Sammenheng med prosjektets overordnede mål

Det overordnede målet for prosjektet er å utvikle en ny kollektivtransport-applikasjon som gir enklere og mer tilgjengelig informasjon, tilbyr sanntidsdata for bedre punktlighet, og har et brukervennlig grensesnitt med støtte for flere språk.

Sprintmålet bidrar direkte til dette ved å:

- **Brukerautentisering og sikkerhet:** Gir en trygg og pålitelig opplevelse, og er en forutsetning for brukertilpassede funksjoner som billettering.
- **Ruteplanlegging:** Gjør det mulig for brukeren å raskt og enkelt finne riktig buss, og adresserer dagens utfordringer med kompliserte grensesnitt.
- **Digital billettering:** Gjør applikasjonen mer helhetlig ved å samle ruteinformasjon og billett kjøp på ett sted.
- **Dummy-data:** Gir et nødvendig testgrunnlag for tidlig utvikling og validering av funksjonalitet.
- **Sanntidsinformasjon:** Løser et sentralt problem i dagens løsning – mangelen på forsinkelsesdata.

7.2 Sprint backlog

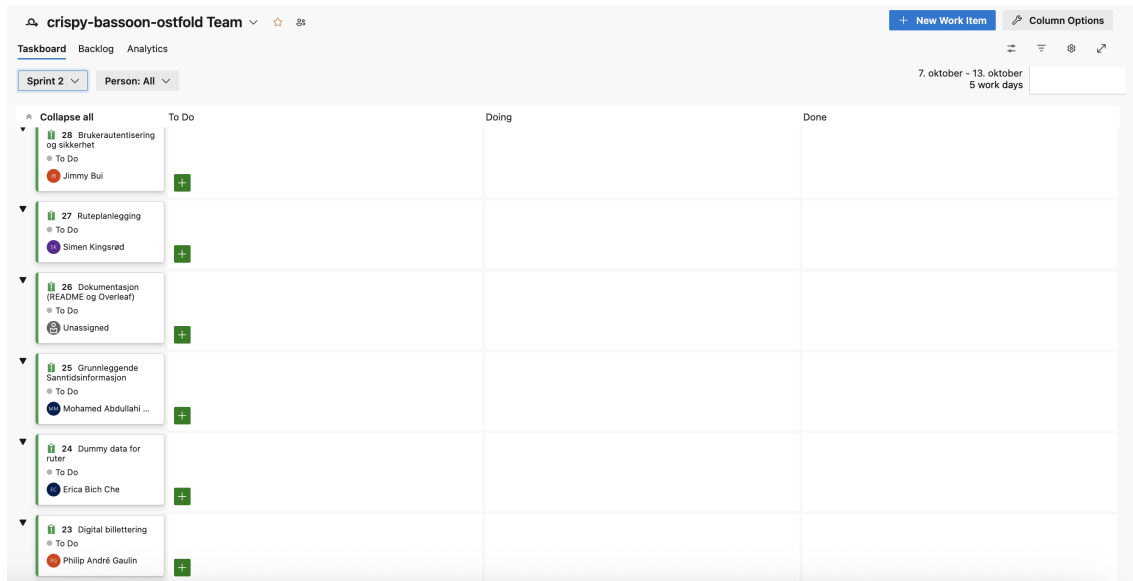
Sprint backlog for Sprint 2 ble basert på hovedmålene over og bygget videre på fundamentet fra Sprint 1. Oppgavene ble loggført i Jira og visualisert som backlog i et kanban-board.

ID	Title	Assigned To	State	Area Path
28	 Brukerautentisering og sikkerhet	 Jimmy Bui	● To Do	crispy-bassoon-ostfold
27	 Ruteplanlegging	 Simen Kingsrød	● To Do	crispy-bassoon-ostfold
25	 Grunnleggende Sanntidsinformasjon	 Mohamed Abdullahi Mah...	● To Do	crispy-bassoon-ostfold
24	 Dummy data for ruter	 Erica Bich Che	● To Do	crispy-bassoon-ostfold
23	 Digital billettering	 Philip André Gaulin	● To Do	crispy-bassoon-ostfold

Figur 7.1: Backlog for Sprint 2

7.3 Kanban-board

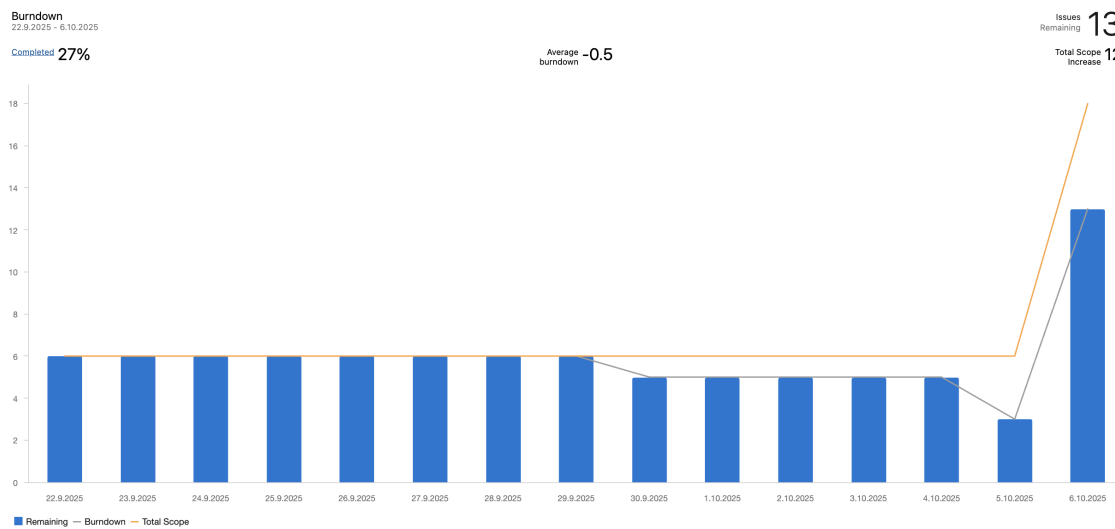
Vi fortsatte med samme arbeidsflyt som i Sprint 1, med et kanban-board strukturert i kolonnene *To Do*, *In Progress*, *Testing/Review* og *Done*.



Figur 7.2: Kanban-board for Sprint 2

7.4 Burndown-chart

Fremdriften i sprinten ble visualisert gjennom et burndown-chart, på samme måte som i Sprint 1.



Figur 7.3: Burndown chart for Sprint 2

7.5 Database og dummydata

Arbeidet med dummydata i Sprint 2 ble gjennomført i tre steg: (1) utvidelse av databaseskjemaet, (2) innlegging av realistiske testdata, og (3) verifisering gjennom SQL-tester i pgAdmin. Målet var å etablere et stabilt og realistisk datagrunnlag for videre utvikling av ruteplanlegging, billettering og sanntidsinformasjon.

Databasen ble utvidet med et nytt **transit**-skjema som inneholder tabeller for ruter, holdeplasser, rekkefølge, turer og rutetider (**line**, **stop**, **line_stop**, **trip**, **stop_time**). Hver rute ble definert med både **OUTBOUND**- og **INBOUND**-retning, og det ble lagt inn representative avganger for morgen, ettermiddag og kveld på tre dagtyper (**WKD**, **SAT**, **SUN**).

Totalt ble tolv ruter (V9–V20) lagt inn, inkludert både lokale og regionale forbindelser, som Fredrikstad–Sarpsborg (linje 1), Halden–Kalnes (linje 632) og Moss–Halden (linje 633). Alle seed-filer ble gjort idempotente, slik at de kan kjøres flere ganger uten å opprette duplikater. Dette sikrer et konsistent og gjenkjørbart datagrunnlag i utviklingsfasen.

Etter innlegging av dummydata ble det gjennomført flere SQL-tester i pgAdmin for å verifisere dataintegriteten. Figurene under viser et utvalg av resultatene fra disse testene.

1	SELECT id, number, name
2	FROM line
3	ORDER BY number;

Data Output	Messages	Notifications
-------------	----------	---------------

									SQL
--	--	--	--	--	--	--	--	--	-----

	id [PK] integer	number text	name text
1	1	1	Fredrikstad–Sarpsborg
2	6	12	Sandbakken–Sykehuset Østfold Kal...
3	7	13	Blessom–Sykehuset Østfold Kalnes
4	8	14	Rekustad–Ise stasjon
5	2	2	Fredrikstad–Sarpsborg
6	9	34	Sykehuset–Remmen
7	4	4	Fredrikstad–Kalnes
8	10	630	Moss–Halden via Fredrikstad
9	11	631	Ørje–Halden via Aremark
10	12	632	Halden–Sykehuset Østfold Kalnes
11	13	633	Moss–Sarpsborg–Halden
12	14	634	Halden–Rakkestad–Mysen

Figur 7.4: Verifisering av at alle busslinjer ble opprettet i tabellen `line`.

Query

Query History

```

1 SELECT l.number, COUNT(st.*) AS antall_tider
2 FROM stop_time st
3 JOIN trip t ON t.id = st.trip_id
4 JOIN line l ON l.id = t.line_id
5 GROUP BY l.number
6 ORDER BY l.number;

```

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	number	antall_tider
	text	bigint
1	1	36
2	12	189
3	13	126
4	14	189
5	2	144
6	34	243
7	4	171
8	630	162
9	631	144
10	632	90
11	633	207
12	634	234

Figur 7.5: Kontroll av at rutetidene er registrert for alle stopp i `stop_time`.

Query

Query History

1

2

3

4

SELECT linje_nr, direction, service, COUNT(*) AS rader
FROM vw_rutetider
GROUP BY linje_nr, direction, service
ORDER BY linje_nr, direction, service;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	linje_nr text	direction direction	service service_day	rader bigint
4	12	OUTBOU...	SAT	39
5	12	OUTBOU...	SUN	39
6	12	INBOUND	WKD	24
7	12	INBOUND	SAT	24
8	12	INBOUND	SUN	24
9	13	OUTBOU...	WKD	21
10	13	OUTBOU...	SAT	21
11	13	OUTBOU...	SUN	21
12	13	INBOUND	WKD	21
13	13	INBOUND	SAT	21
14	13	INBOUND	SUN	21
15	14	OUTBOU...	WKD	24
16	14	OUTBOU...	SAT	24
17	14	OUTBOU...	SUN	24
18	14	INBOUND	WKD	39
19	14	INBOUND	SAT	39
20	14	INBOUND	SUN	39
21	2	OUTBOU...	WKD	24
22	2	OUTBOU...	SAT	24
23	2	OUTBOU...	SUN	24
24	2	INBOUND	WKD	24
25	2	INBOUND	SAT	24
26	2	INBOUND	SUN	24
27	34	OUTBOU...	WKD	42
28	34	OUTBOU...	SAT	42
29	34	OUTBOU...	SUN	42
30	34	INBOUND	WKD	39
31	34	INBOUND	SAT	39
32	34	INBOUND	SUN	39

Figur 7.6: Test av viewet **vw_rutetider** som viser ruter, stopp og klokkeslett samlet.

- Alle ruter ble registrert med riktig antall stopp og korrekt rekkefølge.
- Hver rute fikk turer for alle tre dagtyper og begge retninger.
- **vw_rutetider**-visningen returnerte komplette rutetabeller med korrekte klokkeslett.

Resultatet er en fullstendig fungerende testdatabase som danner grunnlaget for videre backend-utvikling og visning av ruter og avganger i frontend-applikasjonen. Dette legger til rette for integrasjon med sanntidsdata og billetteringssystem i senere sprinter.

7.6 Brukerautentisering og sikkerhet

Vi har videreutviklet autentiseringsløsningen til et komplett rollebasert system med tilgangskontroll. Løsningen støtter fire brukerroller med ulike rettigheter, og er bygget med fokus på sikkerhet, struktur og utvidbarhet.

7.6.1 Database og datamodell

Vi opprettet følgende tabeller i PostgreSQL:

- **role** – lagrer roller.
- **"user"** – lagrer brukerinformasjon og krypterte passord.
- **user_role** – kobling mellom brukere og roller.
- **audit_log** – lagrer logger over viktige handlinger.

Tabellene ble opprettet via Flyway-migrasjoner, og vi la inn fire testbrukere – én per rolle – slik at systemet kunne testes raskt underveis.

7.6.2 Backend og sikkerhet

Vi implementerte et flerlags system bestående av modeller, repositories, tjenester og kontrollere:

- **AuthService** håndterer registrering og innlogging med *BCrypt*.
- **JwtService** genererer og validerer JWT-tokens som varer i 24 timer.
- **SecurityConfig** og **JwtAuthenticationFilter** sørger for at brukere med gyldige tokens og riktige roller får tilgang.

Vi har opprettet egne API-endepunkter for hver rolle:

- **Vanlig bruker:** Kjøpe og se billetter, se profil.
- **Ansatt:** Rapportere og oppdatere avganger.
- **Administrator:** Administrere brukere og se statistikk.
- **Utvikler:** Lese systemlogger og sjekke databasehelse.

I tillegg er det gjort flere forbedringer knyttet til Entur-integrasjon og søkefunksjonalitet:

- **Entur-integrasjon:** Lagt til sanntidsdata (StopPlace + Quay) og visning av forsinkelsesminutter.
- **Søkegrensesnitt:** Implementert et enkelt søk levert fra backend.
- **Stopp-søk:** Lagt til autofullføring via Entur Geocoder med GraphQL som fallback.
- **REST-endepunkter:** Eksponert API-er for søk og avgangsinformasjon.

7.6.3 Bussruter REST-endpoint

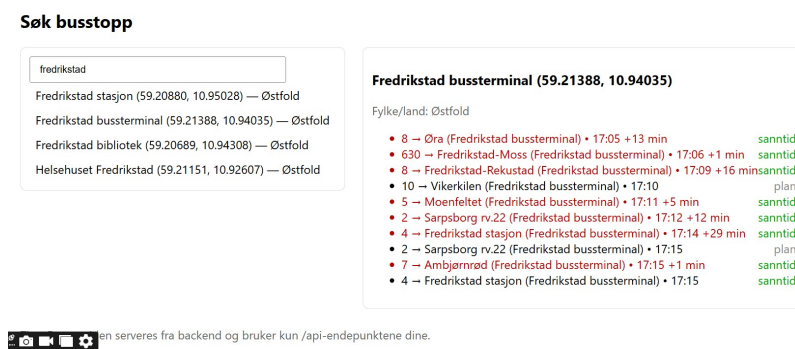
En fungerende REST-endpoint for bussruter ble utviklet som håndterer forespørsler til /api/route, sjekker at dataene er gyldige, og returnerer avganger med tid, varighet og rutenavn. Dette gjør det mulig for frontend å vise avganger og ruter direkte.

Implementert funksjonalitet:

- Opprettelse av en Route-klasse for å lagre informasjon om bussruter.
- REST-endpoint /api/route opprettet i RouteController.
- Generering av eksempel-ruter med avgangs og ankomsttider.
- Returnering av JSON-data til frontend for visning av ruter.

7.6.4 Resultater

- Fungerende autentiseringssystem med JWT-token.
- Fire definerte brukerroller med hierarkisk tilgang.
- Full passordkryptering og sikker innlogging.
- Audit-logging for sporing av handlinger.
- Testdata med fire brukere (én per rolle).
- Sanntidsintegrasjon og søk mot Entur-data.



Figur 7.7: Her er et bilde av backend-endepunktet som viser ulike stopp med autosøk, XY-koordinater, om bussen er i rute, samt hvor mange minutter den eventuelt er forsinket

7.6.5 bildet av sanntidsinformsjajon

7.7 Infrastruktur og deployment

7.7.1 CI/CD-pipeline og deployment

Gjennom Sprint 2 er deployment-pipelinen videreutviklet med blant annet:

- `deploy.sh` – hovedscript for deployment med health checks og feilhåndtering.
- `auto-deploy.sh` – kontinuerlig integrasjon med commit-hash tracking.
- `webhook.php` – webhook mot GitHub Actions for automatisk triggering.
- `manage-services.sh` – håndtering av backend-tjenester via systemd.
- `health-check.sh` – automatisk overvåking av frontend- og backend-endepunkter.

7.7.2 Produksjonsinfrastruktur

Applikasjonen hostes på `itstud.hiof.no/~philipag`, med:

- Frontend deployet til `$HOME/htdocs/app/`.
- Backend som persistent tjeneste på port 8081.
- HTTPS via institusjonens nginx.
- Logging til `$HOME/logs/`.

Produksjonstjenesten er tilgjengelig på `https://itstud.hiof.no/~philipag/app/`.

7.7.3 Utfordringer og læringspunkter

En kritisk utfordring oppstod med routing av API-forespørsler. Nginx-konfigurasjonen aksepterte trafikk til `itstud.hiof.no/~philipag`, men ikke til underkatalogen `/app`, noe som hindret korrekt videresending av forespørsler til backend.

Problemet viste seg i flere lag:

- **API-routing:** Frontend nådde ikke `/auth/login` og `/auth/register`.
- **CORS:** Selv med korrekt Spring-config ble spørringer blokkert pga. nginx-oppsett.
- **Proxy:** Vite-proxy fungerte kun i utviklingsmiljø.

Dette ga stort læringsutbytte innen nginx, reverse proxy og DevOps, men tok også mer tid enn planlagt.

7.8 Retrospekt Sprint 2

Hva gikk bra

Sprint 2 ga tydelig fremgang og etablerte en stabil teknisk plattform. Teamet fikk på plass flere kjernekomponenter, inkludert:

- utvidet database med realistiske dummydata
- rollebasert autentisering
- videreutviklet CI/CD-pipeline

Arbeidsfordelingen mellom frontend og backend fungerte stort sett godt, og flere leveranser ble ferdigstilt innen sprintfristen.

Hva kunne vært bedre

Noen oppgaver viste seg å være større enn anslått, særlig ruteplanlegging og billettering, som krevde tett koordinering mellom database, backend og frontend. Dokumentasjon ble tidvis nedprioritert, og kommunikasjonen kunne vært mer jevn, spesielt rundt API-enderpunkter og datatilgang.

Tiltak og læringspunkter

- Dele opp større oppgaver i mindre backlog-items (1–2 dager).

- Ha korte, faste statusmøter for å fange opp blokkeringer tidlig.
- Oppdatere dokumentasjon fortløpende.
- Tydeliggjøre ansvar direkte i sprintboardet.

Perspektiv fra infrastruktur-ansvarlig

Fokuset på infrastruktur ga verdifull erfaring med:

- systemd service management på låst Linux-miljø
- nginx-konfigurasjon og reverse proxy
- bygging av CI/CD-pipeline fra bunnen av

Samtidig ser vi i etterkant at nginx-problematikken tok mer tid enn planlagt. En tydeligere tidsavgrensning og eventuell pivot til enklere hosting-løsning kunne frigjort tid til mer funksjonell utvikling (for eksempel billettering). Dette tas med som viktig læringspunkt inn i Sprint 3.

Kapittel 8

Sprint 3 – Planlegging og gjennomføring

8.1 Sprintmål

Målet med Sprint 3 er å videreutvikle kjernefunksjonaliteten i applikasjonen og sikre kommunikasjon mellom systemene. Dette innebærer å koble backend mot databasen, implementere repository-klasser for henting av data, og etablere grunnleggende frontend-komponenter for nettsiden og ruteplanlegging. Sprinten markerer et viktig steg fra teknisk grunnstruktur til funksjonell brukeropplevelse, der data nå skal flyte fra database til frontend via REST-API.

Gjennomførte oppgaver i Sprint 2

- Opprettet og fylt databasen med dummy-data for ruter
- Implementert CI/CD-pipeline med kodevalidering
- Ferdigstilt innloggings- og registreringssystem i Java
- Utviklet frontend for innlogging og registrering
- Konfigurert REST-API med CORS-oppsett
- Deployert applikasjonen på utviklingsserver

8.2 Planlegging av Sprint 3

Basert på erfaringene fra Sprint 2 skal Sprint 3 fokusere på å koble sammen de ulike systemkomponentene og legge grunnlaget for videreutvikling av sanntids- og billetteringsfunksjoner. Oppgavene er brutt ned slik at de kan løses innenfor sprintperioden og bidra til jevn fremdrift.

Hovedmål for Sprint 3

- Opprette og teste database-tilkobling
- Implementere repository-klasser for rutedata
- Koble backend til frontend for visning av reelle data
- Utvikle frontend for ruteplanlegging og nettside-layout
- Oppdatere dokumentasjon i README og Overleaf

Sammenheng med prosjektets overordnede mål

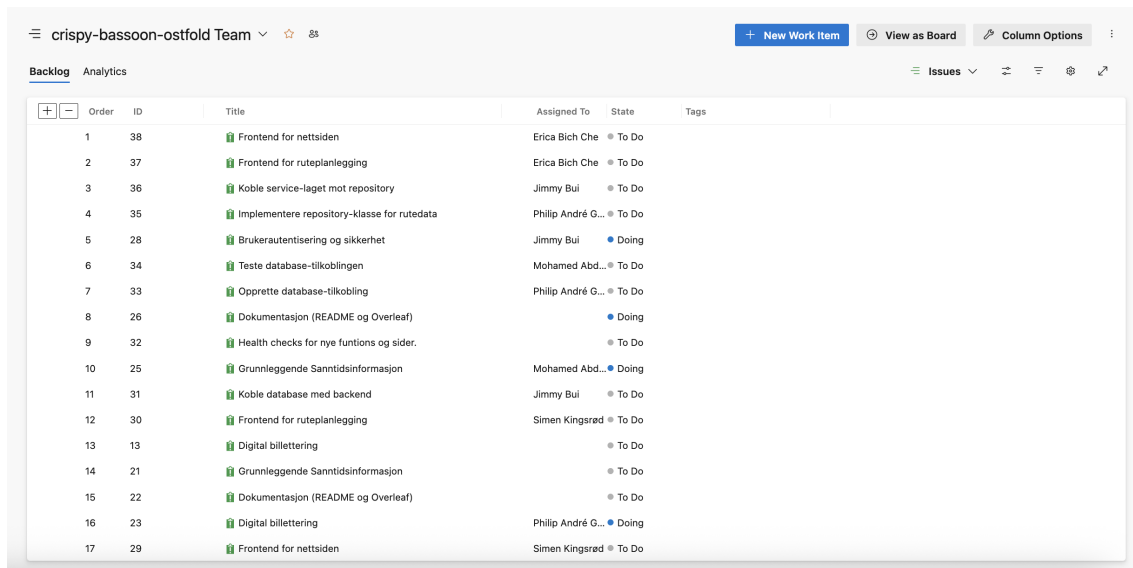
Det overordnede målet for prosjektet er å utvikle en ny kollektivtransport-applikasjon som tilbyr brukervennlig ruteinformasjon, digital billettering og sanntidsdata. Sprint 3 støtter dette målet ved å:

- **Kobling mellom database og backend:** Gjør det mulig å hente og vise dynamiske data for ruter og holdeplasser.
- **Repository- og service-lag:** Gir en strukturert og utvidbar backend-arkitektur som forenkler videre utvikling.
- **Frontend for ruteplanlegging:** Lar brukeren søke og visualisere ruter direkte i applikasjonen.
- **Frontend for nettsiden:** Etablerer navigasjon og layout som danner grunnlag for fremtidige funksjoner.
- **Dokumentasjon:** Bidrar til bedre kommunikasjon i teamet og gjør systemet enklere å vedlikeholde.

Sprint 3 bidrar dermed til å forene de ulike delene av systemet til en fungerende helhet, og legger grunnlaget for videre arbeid med billettering, sanntidsdata og testing i kommende sprinter.

8.3 Sprint backlog

Sprint backlog for Sprint 3 ble basert på hovedmålene for sprinten og bygget videre på arbeidet fra Sprint 2. Oppgavene ble loggført i Jira og visualisert gjennom backlogvisningen.

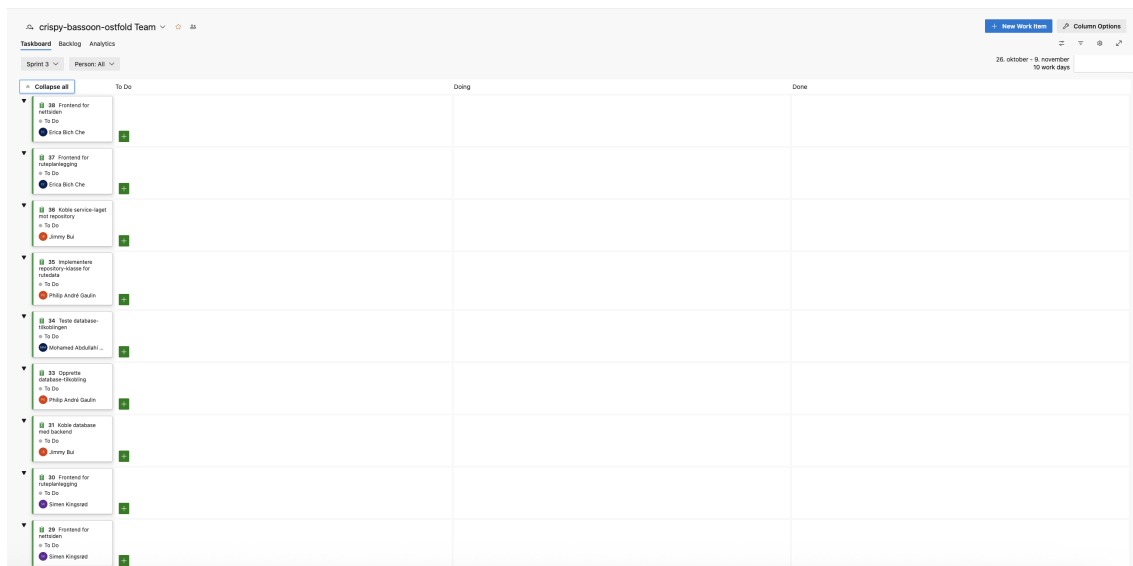


Order	ID	Title	Assigned To	State	Tags
1	38	Frontend for nettsiden	Erica Bich Che	To Do	
2	37	Frontend for ruteplanlegging	Erica Bich Che	To Do	
3	36	Koble service-laget mot repository	Jimmy Bui	To Do	
4	35	Implementere repository-klasse for rutedata	Philip André G...	To Do	
5	28	Brukerautentisering og sikkerhet	Jimmy Bui	Doing	
6	34	Teste database-tilkoblingen	Mohamed Abd...	To Do	
7	33	Opprette database-tilkobling	Philip André G...	To Do	
8	26	Dokumentasjon (README og Overleaf)		Doing	
9	32	Health checks for nye funtions og sider.		To Do	
10	25	Grunnleggende Sanntidsinformasjon	Mohamed Abd...	Doing	
11	31	Koble database med backend	Jimmy Bui	To Do	
12	30	Frontend for ruteplanlegging	Simen Kingsrød	To Do	
13	13	Digital billettering		To Do	
14	21	Grunnleggende Sanntidsinformasjon		To Do	
15	22	Dokumentasjon (README og Overleaf)		To Do	
16	23	Digital billettering	Philip André G...	Doing	
17	29	Frontend for nettsiden	Simen Kingsrød	To Do	

Figur 8.1: Backlog for Sprint 3

8.4 Kanban-board

Arbeidsflyten i Sprint 3 fulgte samme struktur som i de foregående sprintene, med et kanban-board organisert i kolonnene *To Do*, *In Progress* og *Done*.

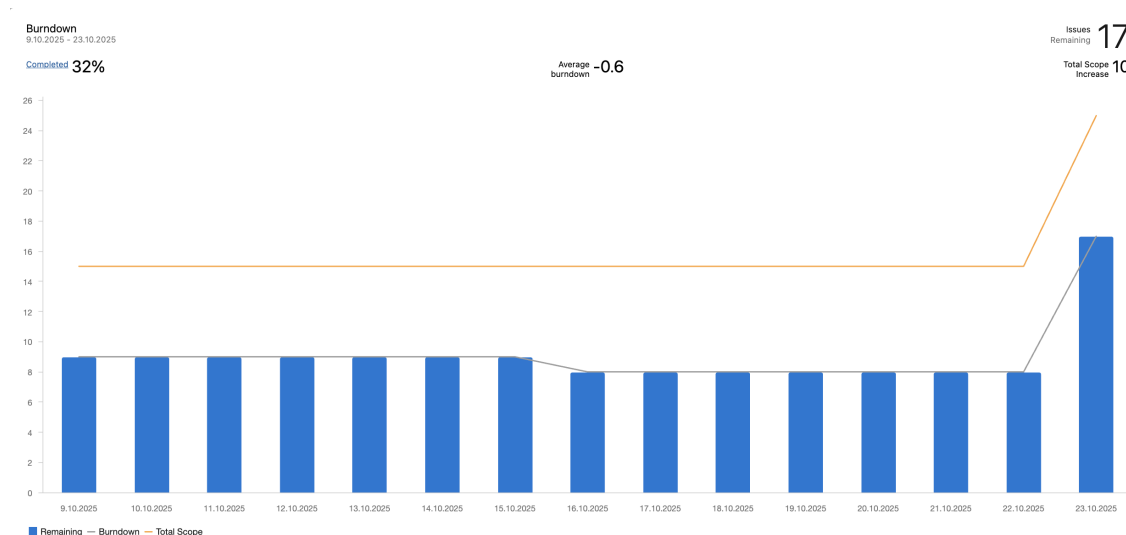


Task	Assigner	State
38 Frontend for nettsiden	Erica Bich Che	To Do
37 Frontend for ruteplanlegging	Erica Bich Che	To Do
36 Koble service-laget mot repository	Jimmy Bui	To Do
35 Implementere repository-klasse for rutedata	Philip André Gaulin	To Do
34 Teste database-tilkoblingen	Mohamed Abdallah	To Do
33 Opprette database-tilkobling	Philip André Gaulin	To Do
31 Koble database med backend	Jimmy Bui	To Do
30 Frontend for ruteplanlegging	Simen Kingsrød	To Do
29 Frontend for nettsiden	Simen Kingsrød	To Do

Figur 8.2: Kanban-board for Sprint 3

8.5 Burndown-chart

Fremdriften i Sprint 3 ble visualisert med et burndown-chart som viser utviklingen av gjenstående arbeid gjennom sprintperioden.



Figur 8.3: Burndown chart for Sprint 3

8.6 Databaseintergrasjon: Backend til PostgreSQL

8.6.1 Oversikt

I sprint 3 fikk backend en kobling mot PostgreSQL-database. Dette inkluderer konfigurasjon av datakilde, oppsett av JPA/Hibernate og etablering av et eget databaseskjema som brukes av applikasjonen.

8.6.2 Teknologivalg

Backend prosjektet bruker:

- Springdata JPA for database operasjoner og ORM.
- PostgreSQL JDBC-driver for kommunikasjon med databasen
- H2 for lokal utvikling og testing
- Flyway for fremtidig versjonskontroll av skjemaer

8.6.3 Konfigurasjon

Databaseoppsettet ligger i application.yml og sikter mot en ekstern PostgreSQL-instans. Spring Boot bruker HikariCP til connection pooling, med et lite og kontrollert antall aktive tilkoblinger. Backend bruker et eget skjema kalt app, og alle tabeller og entiteter ligger der. SQL-logging er også aktivert for lettere feilsøking i utviklingen.

8.6.4 Databasestruktur

Skjemaet defineres i schema.sql og setter opp:

- Et eget app-skjema.
- Roller for administrasjon og vanlige brukere.
- Enum-typer for statuser.
- Tabeller for brukere, stopp, billetter, avganger og ruter.

Strukturen er laget for å være enkel å utvide og håndtere gjennom SQL.

8.6.5 Persistentlag

Entiteter i Java er koblet direkte til tabellene gjennom JPA annotasjoner. Repository-lagret bruker Sprint Data JPA, det gir ferdige CRUD operasjoner og automatiske genererte spørringer som findByEmail ().

8.6.6 Sikkerhet

Passord ligger midlertidig i konfigurasjonsfilen (skal flyttes til miljøvariabler). Spring Data sørger for trygge parameter spørringer, og løsningen kan senere utvides med SSL tilkobling mot databasen.

8.6.7 Oppsummering

I koblingen for database mot backend, etablerte det en stabil og ryddig databaseintergrasjon med PostgreSQL. Backend bruker et eget skjema, klar struktur i databasen og et enkelt persistenslag gjennom JPA. Løsningen er enkelt å vedlikeholde, og klar for å utvikles videre.

8.7 Frontend - ruteplanlegging og nettside-layout

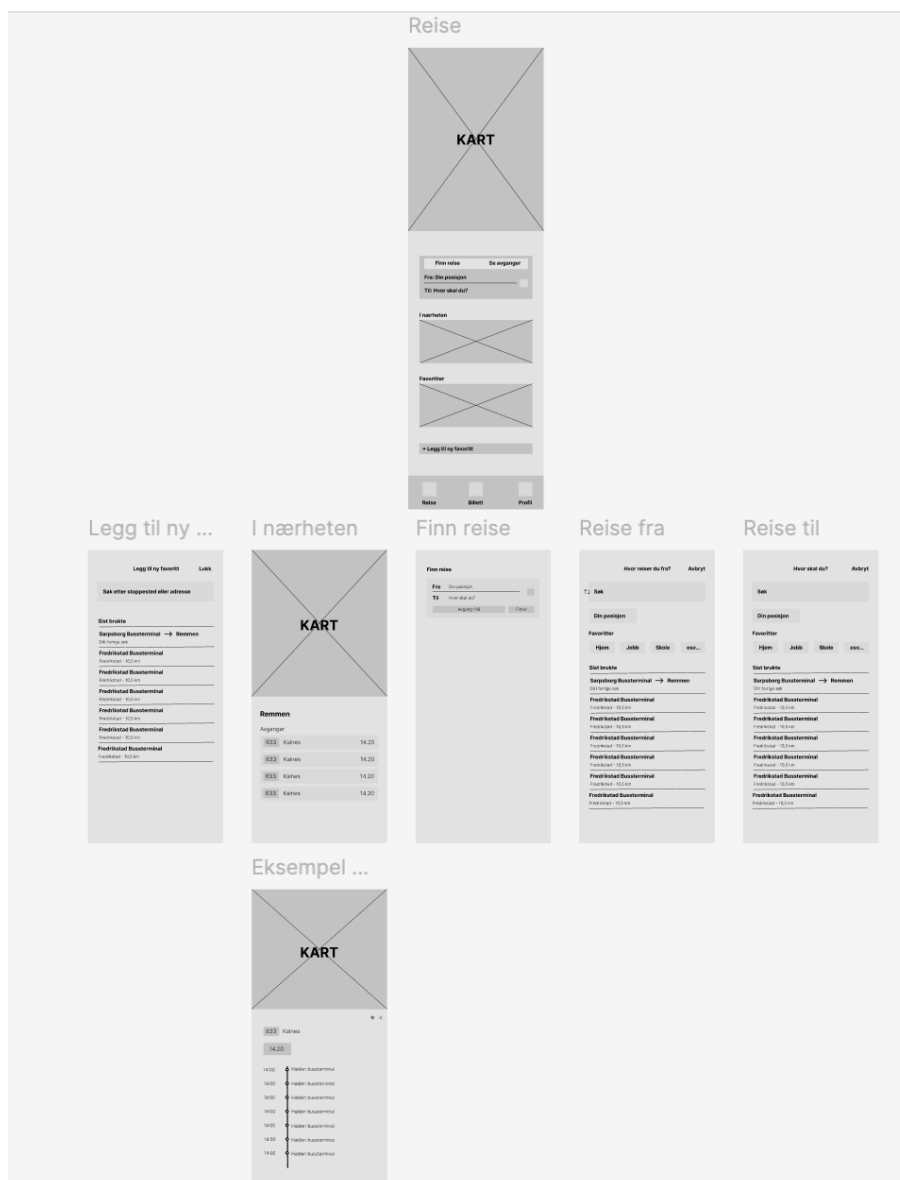
I Sprint 3 ble det arbeidet med å utvikle grunnleggende frontend-funksjonalitet for ruteplanleggingsdelen av applikasjonen. Arbeidet inkluderte:

- etablering av HTML-, CSS og JS-struktur for reise, billett og profil
- Utvikling av layout og visuell utforming med CSS
- implementasjon av Javascript-logikk for UI-interaksjoner (billettvalg, panel og navigasjon)
- Klargjøre komponenter slik at de senere kan kobles til backend-API

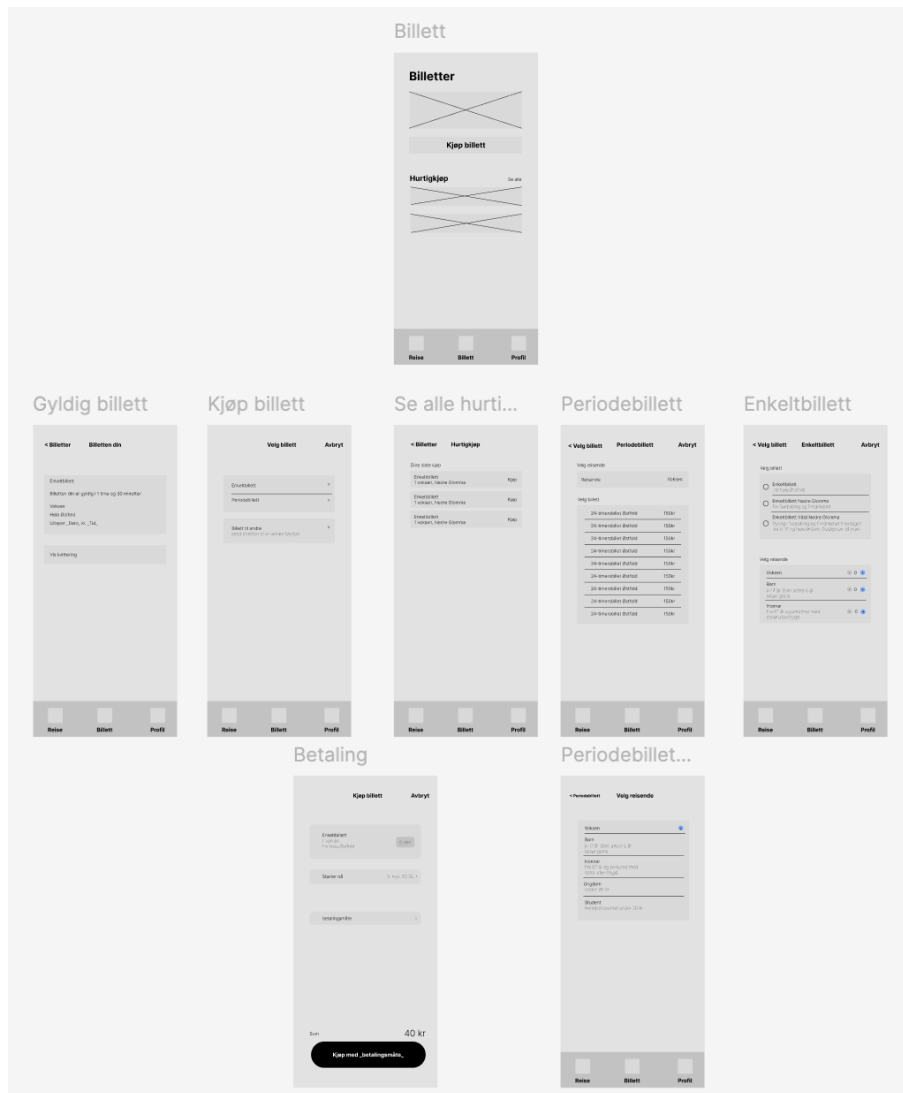
Wireframes for ruteplanlegging

Som en del av designprosessen ble det laget wireframes i Figma for å visualisere struktur og navigasjon før implementasjon. Disse fungerte som en tidlig modell av grensesnittet og sikret en felles forståelse i gruppen.

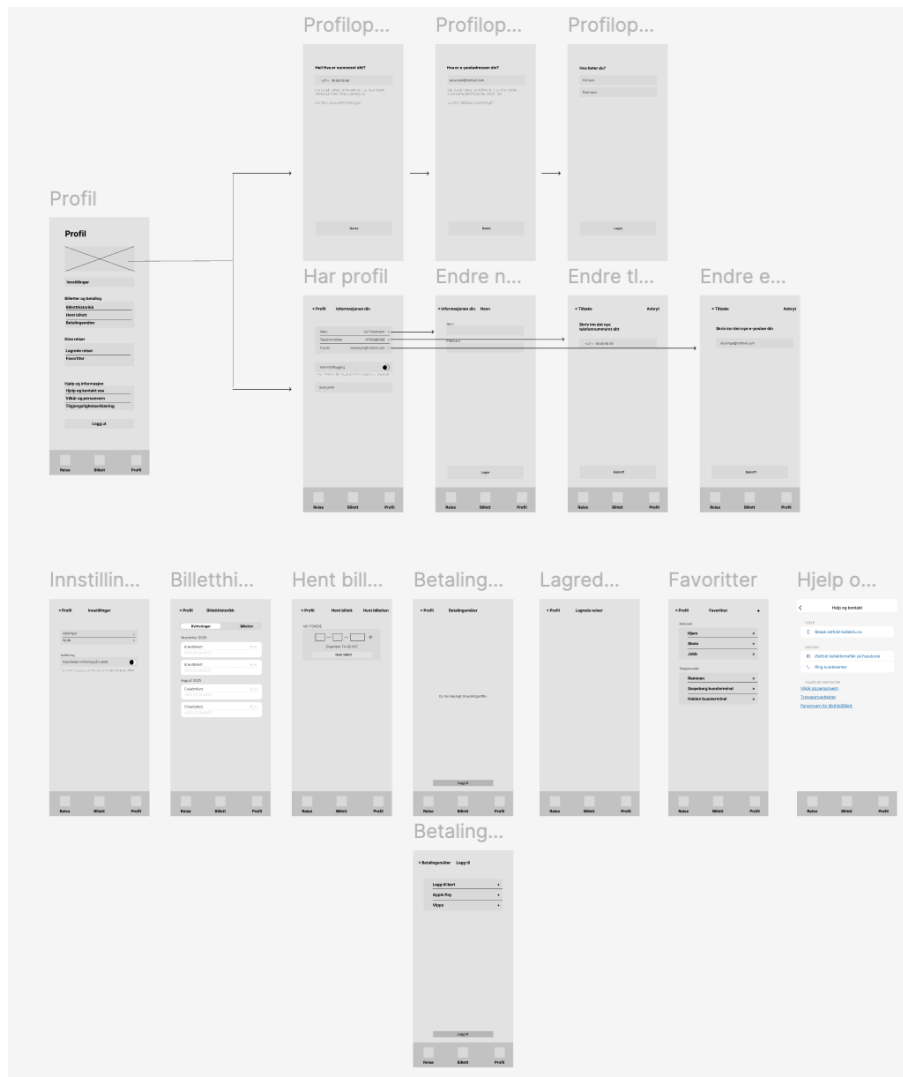
Figurene under viser de viktigste wireframes som ble lagd:



Figur 8.4: Wireframe av hovedsiden for ruteplanlegging.



Figur 8.5: Wireframe av billettoversikt med panelvisning.



Figur 8.6: Wireframe av profilsiden med innstillinger og navigasjon.

Wireframene var et viktig verktøy for å planlegge layout, tekstplassering, ikonbruk og navigasjonsflyt. Dette gjorde frontend-implementasjonen i Sprint 4 betydelig mer effektiv.

Frontend-arbeidet la grunnlaget for et mer komplett ruteplanleggingsgrensesnitt som skal integreres med backend.

De viktigste prioriteringene for Sprint 4 er:

- **Frontend-forbedringer:** Ferdigstille ruteplanleggingsgrensesnittet og sikre at layout, navigasjon og visning av informasjon fungerer helhetlig på tvers av sidene.
- **Utvidelse av nettside-design:** Videreutvikle HTML-, CSS- og JavaScript-strukturene for reise, billett og profil, samt sikre en mer konsistent visuell identitet.

- **Forberedelse for backend-integrasjon:** Strukturere og klargjøre frontend-komponenter slik at de enkelt kan kobles til API-endepunkter i neste fase.
- **Kodeforbedringer og opprydding:** Optimalisere og rydde i eksisterende frontend-kode for å gjøre den mer vedlikeholdbar og oversiktlig.
- **Dokumentasjon:** Oppdatere Sprint-dokumentasjon, README og generell prosjektdokumentasjon for å holde oversikten tydelig og oppdatert.

Sprint 4 fokuserer dermed på å ferdigstille og forbedre frontendløsningen, etablere en mer komplett brukeropplevelse og sikre at systemet er klart for videre integrasjon mot backend-funksjonalitet. Målet er å levere en stabil og visuelt helhetlig versjon av applikasjonens brukergrensesnitt.

8.8 Retrospektiv

Hva gikk bra

- Sprintplanleggingen var mer presis sammenlignet med tidligere sprinter, og oppgavene var bedre fordelt.
- Backlog og kanban-board ble brukt aktivt, noe som ga god oversikt over fremdriften.
- Frontend-utviklingen for ruteplanleggingsdelen gikk som planlagt, og grunnstrukturen i HTML, CSS og JavaScript ble etablert.
- Koblingen mellom backend og databasen kom et godt stykke på vei, og konfigurasjonen mot PostgreSQL fungerte stabilt.

Hva kunne vært bedre

- Kommunikasjonen i teamet kunne vært bedre, spesielt når oppgaver var avhengige av hverandre.
- Dokumentasjonen ble til tider glemt, noe som skapte ekstra arbeid mot slutten av sprinten.
- Noen av sprintoppgavene var større enn forventet, og de blir derfor tatt med videre til neste sprint.

Forbedringspunkter til neste sprint

- Oppdatere dokumentasjon fortløpende i stedet for etter sprintslutt.
- Flere oppdateringer i gruppesamtalen slik at vi vet hvordan sprinten ligger an.
- Bryte ned større oppgaver i mindre, mer håndterbare deler for å sikre jevnere fremdrift.

Oppsummering

Sprint 3 visert tydelig at prosjektet beveger seg i riktig retning, både teknisk og organisatorisk. Teamet klarte å etablere en solid grunnstruktur på tvers av frontend og backend, samtidig som arbeidet med ruteplanlegging og databasekobling gjorde systemet mer funksjonelt enn tidligere.

Samarbeidet fungerte jevnt over godt, men sprinten avdekket også behovet for tettere kommunikasjon og bedre dokumentasjonsrutiner. Enkelte oppgaver viste seg å være større enn forventet, noe som understreker viktigheten av å bryte ned arbeid i mindre deler.

Erfaringene fra Sprint 3 gir et godt grunnlag for forbedring, og tiltakene som ble identifisert vil bidra til en mer effektiv arbeidsflyt og økt kvalitet i neste og siste sprint.

Kapittel 9

Sprint 4 – Planlegging og gjennomføring

9.1 Sprintmål

Målet med Sprint 4 er å ferdigstille frontendløsningen og sikre at applikasjonen fremstår som en helhetlig, navigerbar og brukervennlig prototype. Der Sprint 3 etablerte grunnstrukturen, fokuserer Sprint 4 på å forbedre design, fullføre ruteplanleggingssidene, optimalisere HTML/CSS/JavaScript og sikre konsistent navigasjon mellom alle visninger.

I denne sprinten ble det også lagt vekt på kodeforbedring, opprydding og dokumentasjon, slik at prosjektet er klar til innlevering.

9.2 Planlegging av Sprint 4

Sprint 4 viderefører uferdige oppgaver fra Sprint 3 og legger hovedvekten på frontend-utvikling og visuell helhet. Backend ble ikke videreutviklet i stor grad i denne sprinten, da teamet prioriterte å ferdigstille brukeropplevelsen i prototypen.

Hovedmål for Sprint 4

- Fullføre frontend for ruteplanlegging og relaterte sider.
- Forbedre nettside-layout og sikre konsistent design.
- Implementere og forbedre JavaScript-funksjonalitet for navigasjon og UI-interaksjoner.
- Rydde og optimalisere kode i HTML, CSS og JS.
- Sikre at alle sider fungerer i sammenheng og at navigasjonen er intuitiv.

- Oppdatere og ferdigstille dokumentasjon.

9.3 Gjennomføring og oppgaver

Backend og integrasjon

- Implementere e-post-basert flyt for **glemt passord**, inkludert tokenvalidering og ny passordside.
- Aktivere endepunkt for **billettrefusjon** med bekreftelsesdialog i UI.
- Implementere **billett nedlasting** som PDF eller QR-kode via `GET /api/tickets/{id}/download`.
- Koble **sanntids-API** (`GET /api/realtime/bus/{busId}`) til kartvisningen for å vise bussposisjoner.

Admin-dashboard

- Implementere egen **admin-innlogging** og rute til dashboard.
- Lage oversikt med seksjoner for **brukere**, **busser**, **ruter** og **systemstatistikk**.
- Koble dashboardet til eksisterende backend-API-er for administrasjon.

Testing og kvalitetssikring

- Utføre **funksjonstesting** av nye moduler: passordreset, billett nedlasting, refusjon og sanntidsoppdateringer.
- Gjennomføre **UI-testing** på mobil, nettbrett og desktop.
- Kontrollere **tilgjengelighet (WCAG)** og responsivitet.
- Oppdatere **README** og dokumentasjon i Overleaf med nye skjermbilder og flytbeskrivelser.

9.4 Frontend – Gjennomført arbeid

Dette kapittelet beskriver arbeidet som ble utført i frontend i løpet av prosjektet, med fokus på forbedring av layout, navigasjon, struktur og visuell helhet i prototypen. Arbeidet har vært sentrert rundt HTML, CSS og JavaScript, der målet har vært å utvikle en ryddig, sammenhengende og brukervennlig løsning.

9.4.1 Forbedring av struktur og kodekvalitet

En vesentlig del av frontend-arbeidet bestod av opprydding og standardisering av koden. Dette inkluderte:

- omorganisering av HTML-struktur for bedre lesbarhet og enklere vedlikehold,
- rydding og reduksjon av unødvendig CSS,
- samling av stilregler for konsistent typografi, marginer, farger og knappelementer,
- forbedring av JavaScript-kode for å gjøre funksjoner mer oversiktlige og modulære,
- fjerning av duplisert kode og forbedring av navngivningskonvensjoner.

Dette arbeidet bidro til en mer profesjonell frontendkodebase som er enklere å bygge videre på.

9.4.2 Designforbedringer og visuelle elementer

For å gi applikasjonen et mer helhetlig uttrykk ble flere designmessige justeringer gjennomført:

- lagt inn nye ikoner for navigasjon og handlinger, slik at grensesnittet ble mer intuitivt,
- forbedret fargepalett og kontraster for bedre lesbarhet,
- oppdatert knappestiler, paneler og header-elementer for visuell konsistens,
- justert spacing, padding og layout for å sikre et renere og mer moderne uttrykk,

Disse endringene ga applikasjonen en tydeligere identitet og en mer profesjonell følelse.

9.4.3 Navigasjon og UI-funksjonalitet

Frontend fikk også funksjonelle forbedringer som gjorde prototypen mer brukervennlig:

- implementasjon av navigasjonslogikk mellom siden for reise, billetter og profil,
- forbedret JavaScript-styring av paneler, animasjoner og visning av innhold,
- justert navigasjonslinje og knapper for å sikre konsistent oppførsel,
- strukturert visning av ruteplanleggingskomponenter.

Dette gjør at applikasjonen kan testes og demonstreres som en sammenhengende løsning.

9.4.4 Engelsk versjon av prototypen

For å gjøre løsningen mer tilgjengelig ble en engelsk versjon av frontenden utviklet. Dette inkluderte:

- oversettelse av knapper, overskrifter og UI-tekst,
- egen HTML-fil med engelsk språkdrakt,
- justering av JS for å sikre at tekst og elementer samsvarte i begge språkversjoner.

Dette gir prototypen mer fleksibilitet og gjør det lettere å vise frem løsningen til et bredere publikum.

9.5 Oppsummering

Frontend-arbeidet i prosjektet har resultert i en mer brukervennlig, oversiktlig og visuelt konsistent prototype. Arbeidet har vært fokusert på ryddighet, designkvalitet og tydelig navigasjon, og gjør at systemet fremstår som et gjennomført produkt selv uten full backend-integrasjon.

9.6 Forventet resultat av Sprint 4

Etter Sprint 4 skal applikasjonen være funksjonelt komplett, med alle hovedfunksjoner operative og testet. Dette inkluderer sikker pålogging, passordhåndtering, billettkjøp, nedlasting, refusjon, sanntidsdata og fullt fungerende innstillingsside. Frontend-designet skal være helhetlig og responsivt, og løsningen skal kunne demonstreres som en stabil og ferdigstilt prototype klar for produksjonsmiljø.

9.7 Utfordringer og læringspunkter

9.7.1 Største utfordringer

Tekniske utfordringer

- Oppsett av React Native og håndtering av cross-platform-kompatibilitet.
- Konfigurasjon av Spring Boot Security med JWT på en sikker og samtidig håndterbar måte.
- Design av database som både støttet funksjonaliteten og ga akseptabel ytelse.
- Konfigurasjon av CI/CD-pipeline for flere språk (Java og TypeScript) og flere miljøer.

9.7.2 Løsningsstrategier

Tekniske tiltak

- Etablering av enkle, men tydelige kodestandarder for både backend og frontend.
- Innføring av testingstrategi med fokus på kritiske funksjoner først.
- Regelmessige arkitektur-gjennomganger for å sikre at løsningene var bærekraftige.
- Dokumentasjon av oppsett og konfigurasjon for å forenkle feilsøking og onboarding.
- Enighet om at all backlog, commits og tekniske kommentarer skrives på engelsk for å være konsekvente.

9.8 Resultat og refleksjon

9.8.1 Leveranse mot plan

Vi hadde som mål å levere alle kritiske MVP-funksjoner innen prosjektfristen. Noen *nice-to-have*-funksjoner ble bevisst nedprioritert til fordel for robust testing, stabilitet og god dokumentasjon. Dette vurderer vi som en riktig prioritering med tanke på kvalitet og vedlikeholdbarhet.

9.8.2 Teamutvikling

Prosjektet bidro til å utvikle oss fra en gruppe studenter med ulik bakgrunn til et mer samkjørt utviklingsteam. Gjennom bruk av Scrum, felles verktøy og tydelige roller opplevde vi økt eierskap, bedre kommunikasjon og høyere kvalitet på leveransene.

9.8.3 Teknisk gjeld og vedlikehold

Vi forsøkte å begrense teknisk gjeld ved å prioritere ren kode, enhetstester og dokumentasjon fremfor raske, midlertidige løsninger. Dette gjør at systemet lettere kan videreutvikles etter prosjektets slutt, både av oss selv og eventuelle nye utviklere.

Kapittel 10

Status for prosjektet ved innlevering

Dette kapitlet oppsummerer prosjektets status ved innlevering etter gjennomførte fire sprinter. Det gir en helhetlig oversikt over kravoppnåelse, systemdesign, implementasjon, testing og organisering.

10.1 Kravspesifikasjon

Kravspesifikasjonen har blitt oppdatert gjennom hele prosjektet og reflekterer en tydelig prioritering av MVP-funksjonalitet. Følgende hovedkrav er definert og delvis eller helt implementert:

- **F01 – Autentisering:** Implementert med databasekobling, hashing og JWT.
- **F02 – Ruteplanlegging:** Implementert; frontend-struktur ferdig.
- **F03 – Avgangsoversikt:** Delvis implementert.
- **F04 – Billettering:** Backend-endepunkter implementert, frontend-prototype ferdig.
- **F05 – Sanntidsinformasjon:** Backend-funksjonalitet implementert.
- **F06 – Administrasjon:** Admin-endepunkter tilgjengelige.
- **F07 – CI/CD:** Fullt implementert med deploy til produksjonsserver.

Utvidede krav (F08–F10) er dokumentert, men ligger utenfor MVP. Kravene har dannet grunnlaget for sprintplanlegging og prioriteringer.

10.2 Design og modeller

Systemdesignet har utviklet seg gjennom prosjektperioden og består av:

- **Konseptarkitektur** som viser kommunikasjonen mellom frontend, backend og database.
- **Sekvensdiagram** for innlogging, rutesøk og billettkjøp.
- **Datamodell / ER-diagram** med tabeller for brukere, ruter, stopp, avganger, billetter og forsinkelser.
- **Klassediagram** for sentrale backend-moduler.
- **Mockups i Figma** for hovedsider og navigasjon.

Designet viser et modulært system med tydelig separasjon mellom lagene og en fleksibel dataflyt som støtter videre utvikling.

10.3 Prototype og implementasjon

Backend

- Fullt fungerende innloggings- og registreringssystem (JWT, hashing, validering).
- PostgreSQL-database med seed-data for ruter, avganger og brukere.
- Stort REST-API med endepunkter for autentisering, ruter, billetter, sanntid, profil, favoritter, innstillinger og admin.
- Modulbasert backend-struktur (controller, service, repository).
- Full CI/CD-pipeline for bygging, testing og automatisk deploy.

Frontend

- Ferdigstilt grunnstruktur i HTML, CSS og JavaScript.
- Sideoppsett for reise, billetter og profil.
- Implementert ruteplanleggingsvisninger og navigasjon.
- Oppdatert med engelsk og norsk versjon.
- Ryddet, optimalisert og modernisert kodebase.
- Forbedret UI med ikoner, responsivitet og konsistent design.

Database

- Fullstendig skjema med tabeller for brukere, roller, ruter, stopp, avganger, billetter og forsinkelser.
- Seed-data generert og verifisert.
- Kjørbart `schema.sql` og eget `app-skjema`.

Deployment

- Applikasjonen er deployet på: <https://itstud.hiof.no/~philipag/app/>
- Webhook-basert CI/CD for automatisk deploy ved push til **main**.
- Automatiserte server-scripts for bygging, health-check og service restart.

10.4 Testing

Manuell testing

- Innlogging og registrering testet med Postman og frontend.
- Navigasjon og layout testet i frontend-prototypen.
- Kontroll av seed-data og SQL-viewer i pgAdmin.

Automatisk testing

- Pre-commit testing via Git-hooks.
- Automatisk testkjøring i GitHub Actions.
- Health-checks på server hvert 2. minutt.

Enhetstester og integrasjonstester er planlagt for videre arbeid, men ikke fullført innen innlevering.

10.5 Organisering og arbeidsprosess

Prosjektet ble gjennomført etter en Scrum-inspirert arbeidsmetodikk fordelt på fire sprinter. Arbeidet er organisert gjennom GitHub Projects, Discord og fysiske møter.

Rollefordeling

- **Jimmy Bui:** Backend, sikkerhet og autentisering.
- **Erica Bich Che:** Database, datamodell, frontend og hovedansvar for rapport.
- **Simen Kingsrød:** Frontend og ruteplanlegging.
- **Mohamed Abdullahi Mahamed:** sanntidsinformasjon.
- **Philip André Gaulin:** DevOps, billettering, API-integrasjon og deployment.

Sprintrefleksjoner viser forbedret kommunikasjon, tydeligere rollefordeling og en mer strukturert arbeidsflyt gjennom prosjektet.

10.6 Oppsummering

Prosjektet har utviklet seg betydelig gjennom de fire sprintene, og fremdriften har naturlig avveket fra den opprinnelige planen som ble lagt ved semesterstart. Den opprinnelige planen antok en jevn progresjon mellom backend, database og frontend i hver sprint. Etter hvert som arbeidet kom i gang, ble det tydelig at flere oppgaver som backendintegrasjon, deployment og databasekobling var mer komplekse enn forventet. Dette førte til at deler av funksjonaliteten måtte flyttes til senere sprinter.

Sprintrefleksjonene viser tydelig hvordan prosjektet utviklet seg fra å etablere autentisering, database og serverinfrastruktur i Sprint 1–2, til mer helhetlig utvikling av både backend og frontend i Sprint 3–4. Etter hvert som grunnsystemet kom på plass, ble fokuset utvidet til å inkludere ruteplanlegging, billettering, sanntidsdata og en mer komplett brukeropplevelse i frontend.

Endringene i fokus skyldtes både tekniske utfordringer, avhengigheter mellom backend, database og UI, samt behovet for å prioritere funksjonalitet som måtte være på plass før innlevering. Denne kontinuerlige justeringen av retning og prioriteringer er i tråd med Scrum-prinsippene, der teamet lærer og tilpasser seg etter hver sprint for å sikre best mulig fremdrift og kvalitet.

Ved innlevering består prosjektet av en robust backend med omfattende REST-API, komplett databaseoppsett, fungerende autentisering, en ferdig frontend-prototype og en produksjonsklar CI/CD-pipeline. Selv om enkelte opprinnelige planer ble revidert, er MVP-grunnlaget ferdigstilt og prosjektet fremstår som helhetlig, funksjonelt og godt dokumentert.

Kapittel 11

Utviklingsprosess

Dette kapitlet beskriver utviklingsprosessen som førte til den ferdige prototypen. Vi går gjennom de ulike fasene i utviklingen, fra de første enkle eksemplene til en fungerende fullstendig applikasjon. Kapitlet dekker både tekniske valg, verktøy som ble brukt, og utfordringer som oppstod underveis. Vi beskriver også hvordan vi løste problemer og justerte tilnærmingen basert på erfaringer fra utviklingsprosessen.

11.1 Veien til produktet

Utviklingsprosessen ble organisert i fire sprinter over en periode på omtrent åtte uker, hvor hver sprint varte i to uker. Vi fulgte Scrum-rammeverket tilpasset et studentprosjekt, med tydelige roller, sprintplanlegging og retrospektiver. Prosessen startet enkelt med å etablere grunnleggende funksjonalitet i både backend og frontend, og vi begynte med klassiske "Hello World"eksempler for å sikre at utviklingsmiljøet var korrekt konfigurert.

11.1.1 Sprint-basert utvikling

Prosjektet ble organisert i fire hovedfaser:

Sprint 1 fokuserte på å etablere et solid fundament ved å sette opp database med brukertabeller, implementere grunnleggende autentisering i backend, og utvikle enkel frontend for innlogging og registrering. Dette inkluderte også oppsett av CI/CD-pipeline i GitHub Actions.

Sprint 2 bygde videre på fundamentet ved å utvide databasen med realistiske dummy-data for ruter, implementere rollebasert autentisering med JWT-tokens, og videreutvikle CI/CD-pipelinen med automatisk deployment.

Sprint 3 fokuserte på å koble backend mot PostgreSQL-databasen, implementere repository-klasser for henting av rutedata, og utvikle frontend-komponenter for ruteplanlegging og nettside-layout.

Sprint 4 var den siste sprinten hvor vi ferdigstilte frontendløsningen, forbedret design og navigasjon, implementerte billett nedlasting og refusjon, og oppdaterte dokumentasjonen.

11.1.2 Innledende oppsett

Prosessen startet med å sette opp et Spring Boot-prosjekt i Java for backend-delen. Dette innebar å opprette en standard Maven-struktur med de nødvendige avhengighetene for web-applikasjoner, database-tilgang og sikkerhet. Filstrukturen følger standard Maven-konvensjoner med klare separasjoner mellom kildekode, ressurser og tester:

```
src/  
  main/  
    java/  
      no/ostfold/bussapp/  
        auth/  
        ticket/  
    resources/  
      application.properties  
  test/  
    java/  
      no/ostfold/bussapp/
```

Denne strukturen gir god organisering av koden og gjør det enkelt å navigere i prosjektet. Pakkestrukturen reflekterer domenemodellen, med separate pakker for autentisering og billettfunksjonalitet.

For frontend-delen startet vi med et React-prosjekt. I begynnelsen prøvde vi React Native for cross-platform-utvikling, men oppdaget at dette var mer komplekst enn forventet. Vi endret derfor tilnærmingen til å bruke standard React for web-applikasjonen, noe som ga oss raskere fremdrift og bedre kontroll over utviklingsprosessen.

11.1.3 Utfordringer og løsninger

Under utviklingen møtte vi flere utfordringer som krevde justeringer i tilnærmingen. Noen av disse tok mer tid enn forventet, men ga oss verdifull læring.

Spring Security og JWT-tokens

En av de første utfordringene var å forstå hvordan Spring Security fungerte sammen med JWT-tokens. Opprinnelig prøvde vi å implementere autentisering på en måte som ikke passet godt med Spring Security's arkitektur. Etter å ha studert dokumentasjonen og eksempler, endret vi tilnærmingen til å bruke Spring Security's innebygde støtte for JWT-tokens gjennom filter-kjeden. Dette krevde en grundig forståelse av hvordan filter-kjeden fungerer, og hvordan JWT-tokens valideres på hver forespørsel.

Database-operasjoner med PostgreSQL-enums

En annen utfordring oppstod med database-operasjoner. Vi startet med å bruke Spring Data JPA repositories for alle operasjoner, men oppdaget at noen operasjoner, spesielt de som involverte PostgreSQL-enums, var vanskeligere å håndtere med standard JPA. Dette førte til at vi introduserte JdbcTemplate for spesifikke operasjoner, som billett-opprettelse med enum-konvertering:

```
String sql = ""
    INSERT INTO app.ticket (user_id, departure_id, qr_code_hash,
    purchased_at, fare, price_cents, state)
    VALUES (?, ?, ?, ?, ?::fare_type, ?, ?::ticket_state)
    RETURNING id
    """;

Long ticketId = jdbcTemplate.queryForObject(
    sql, Long.class, userId, departureId, qrCodeHash,
    purchasedAt, fareType.name(), priceCents,
    Ticket.TicketState.NEW.name()
);
```

Dette eksemplet viser hvordan vi bruker native SQL med eksplisitt type-casting for PostgreSQL-enums, noe som gir oss full kontroll over database-operasjonene.

Infrastruktur og deployment

En av de største utfordringene oppstod i Sprint 2 med oppsett av produksjonsserveren. Vi opplevde problemer med nginx-konfigurasjonen som hindret korrekt routing av API-forespørsler. Nginx aksepterte trafikk til hoveddomenet, men ikke til underkatalogen /app, noe som hindret korrekt videresending av forespørsler til backend.

Problemet viste seg i flere lag: API-routing fungerte ikke, CORS ble blokkert selv med korrekt Spring-konfigurasjon på grunn av nginx-oppsett, og Vite-proxy fungerte kun i utviklingsmiljø. Dette ga stort læringsutbytte innen nginx, reverse proxy og DevOps, men tok også mer tid enn planlagt. I etterkant ser vi at en tydeligere tidsavgrensning og eventuell pivot til enklere hosting-løsning kunne frigjort tid til mer funksjonell utvikling.

Java 25-kompatibilitet

Under testing oppdaget vi at Mockito ikke støttet Java 25 offisielt, da Byte Buddy (som Mockito bruker) kun støttet opp til Java 22. Dette førte til at testene ikke kunne kjøres. Løsningen var å legge til system property net.bytebuddy.experimental=true i Maven Surefire-plugin konfigurasjonen, som aktiverer eksperimentell støtte for nyere Java-versjoner.

11.1.4 Refaktorering og forbedringer

Etter at vi hadde en fungerende prototype, gikk vi gjennom koden for å forbedre kvaliteten. Dette inkluderte å legge til bedre feilhåndtering, forbedre validering av input, og sikre at koden følger best practices. Vi refaktorerte også noen deler av koden for å gjøre den mer lesbar og vedlikeholdbar.

En viktig forbedring var å implementere graceful error handling i JWT-token validering. Opprinnelig kastet systemet unntak når tokens var utløpt eller ugyldige, men vi endret dette til å returnere boolean-verdier i stedet. Dette gir bedre brukeropplevelse og gjør feilhåndteringen mer forutsigbar. Implementeringen fanger `ExpiredJwtException` og ekstrakter informasjon fra tokenets claims selv etter utløp, noe som gir mulighet for å validere tokenet uten å kaste unntak.

11.2 Verktøy og metoder

Utviklingen av prototypen krevde bruk av flere verktøy og metoder for å sikre kvalitet og effektivitet. Dette inkluderer både utviklingsverktøy, byggeverktøy, og metoder for testing og kvalitetssikring.

11.2.1 Utviklingsverktøy

IntelliJ IDEA ble brukt som primær IDE for backend-utvikling. Dette verktøyet gir god støtte for Spring Boot-utvikling med autocomplete, refaktorering, og integrert debugging. For frontend-utvikling ble Visual Studio Code og Cursor brukt, som gir god støtte for React-utvikling med extensions for syntax highlighting, linting og debugging.

Versjonskontroll ble håndtert gjennom Git, med GitHub som remote repository på github.com/craftpag/crispy-bassoon-ostfold. Dette tillot oss å samarbeide effektivt og holde oversikt over endringer i koden. Vi brukte en GitFlow-inspirert branching-strategi med main som produksjonsbranch, develop som integrasjonsbranch, og feature-branches for hver MVP-funksjonalitet. Alle pull requests måtte godkjennes av minst én annen utvikler, og vi fulgte Conventional Commits for å legge til rette for automatisk changelog-generering.

11.2.2 Byggeverktøy og avhengighetshåndtering

Maven ble brukt for å håndtere avhengigheter og bygge backend-applikasjonen. Maven's dependency management gjør det enkelt å legge til nye biblioteker og sikrer at alle nødvendige avhengigheter inkluderes. Package.json og npm ble brukt for frontend-delen, som er standard for React-prosjekter.

Build-prosessen er automatisert gjennom Maven, som kompilerer koden, kjører tester, og pakker applikasjonen i en JAR-fil som kan kjøres. For frontend genererer npm build-prosessen en optimalisert versjon av applikasjonen som kan serveres statisk eller integreres i backend.

11.2.3 CI/CD og deployment

GitHub Actions ble brukt for å automatisere bygging og testing av applikasjonen. Pipelineen validerer kode ved pull requests, kjører automatiserte tester, og deplojer automatisk til staging-miljø ved merge til develop. Produksjonsdeployment krever manuell godkjenning.

Produksjonsserveren er hostet på itstud.hiof.no/philipag/app/, med frontend deployet til [htdocs/app/](https://itstud.hiof.no/philipag/app/) og backend som persistent tjeneste på port 8081. Deployment håndteres gjennom shell-scripts (`deploy.sh`, `auto-deploy.sh`) som stopper tjenester, henter siste versjon fra Git, installerer avhengigheter, bygger frontend, og starter tjenester på nytt. Scriptet verifiserer også at alle tjenester kjører som forventet gjennom health checks.

11.2.4 Testing og kvalitetssikring

Testing ble implementert ved hjelp av JUnit 5 og Mockito for backend. Dette gir oss mulighet til å teste funksjonalitet isolert uten avhengighet til eksterne systemer som databaser. Testene sikrer at kritiske funksjoner oppfører seg korrekt og håndterer edge cases på riktig måte.

For å sikre at testene kan kjøres med nyere Java-versjoner, måtte vi konfigurere Maven Surefire-plugin med spesielle innstillinger. Dette viser viktigheten av å holde seg oppdatert på kompatibilitetsproblemer mellom verktøy og plattformer.

11.2.5 Database og migrasjoner

Flyway ble brukt for database-migrasjoner, som gir oss kontroll over database-skjemaet og sikrer at endringer kan gjøres på en kontrollert måte. Migrasjonene er versjonert og kan kjøres automatisk når applikasjonen starter, noe som sikrer at databasen alltid er i riktig tilstand.

PostgreSQL 16 ble valgt som database på grunn av god støtte for avanserte datatyper som enums, som vi trengte for billetttyper og billettstatus. Dette gir oss mulighet til å ha type-sikkerhet på database-nivå, ikke bare i applikasjonskoden. Databasen ble administrert gjennom pgAdmin 4, og vi opprettet separate roller (`app_admin` og `app_user`) for skille mellom administrasjon og applikasjonsbruk.

I Sprint 2 ble databasen utvidet med et transit-skjema som inneholder tabeller for ruter, holdeplasser, rekkefølge, turer og rutetider. Totalt ble tolv ruter lagt inn med representative avganger for morgen, ettermiddag og kveld på tre dagtyper (WKD, SAT, SUN). Alle seed-filer ble gjort idempotente, slik at de kan kjøres flere ganger uten å opprette duplikater.

11.2.6 API-utvikling

RESTful API ble designet med fokus på klarhet og konsistens. Endepunktene følger REST-konvensjoner med tydelige HTTP-metoder og statuskoder. API-et er dokumentert for å gjøre det enkelt å forstå hvordan det skal brukes.

Sikkerhet ble implementert gjennom JWT-tokens, som gir stateless autentisering. Dette gjør systemet skalerbart og enkelt å integrere med frontend-applikasjonen. Tokens inneholder nødvendig informasjon for å identifisere brukeren og valideres på hver forespørsel.

11.3 Refleksjoner over utviklingsprosessen

Utviklingsprosessen ga verdifull erfaring med moderne web-utvikling og fullstack-applikasjoner. Som studenter med varierende bakgrunn var det viktig å etablere en struktur som både støttet læring og sikret jevn fremdrift mot MVP-målet.

11.3.1 Læringspunkter

En viktig læring var viktigheten av å starte enkelt og bygge gradvis, i stedet for å prøve å implementere alt på en gang. Dette tillot oss å lære og justere tilnærmingen underveis. Sprint-basert utvikling med tydelige mål og retrospektiver gjorde det mulig å reflektere over hva som fungerte og hva som kunne forbedres.

En annen viktig læring var viktigheten av testing tidlig i prosessen. Ved å skrive tester samtidig med funksjonaliteten, oppdaget vi problemer tidligere og kunne løse dem før de ble større. Dette gjorde også refaktorering tryggere, siden vi kunne verifisere at endringene ikke brøt eksisterende funksjonalitet.

Prosessen viste også viktigheten av å være fleksibel og villig til å endre tilnærmingen når man oppdager bedre løsninger. Noen av de første implementasjonene ble endret betydelig etter hvert som vi fikk mer forståelse for problemet og de tilgjengelige løsningene. For eksempel endret vi fra React Native til standard React da vi oppdaget at cross-platform-utvikling var mer komplekst enn forventet.

11.3.2 Organisatoriske utfordringer

Kommunikasjonen i teamet kunne vært bedre, spesielt når oppgaver var avhengige av hverandre. Noen oppgaver viste seg å være større enn forventet, og dokumentasjonen ble tidvis nedprioritert. Dette førte til at vi måtte ta med noen oppgaver til neste sprint. Vi lærte viktigheten av å bryte ned større oppgaver i mindre deler, ha hyppigere statusoppdateringer, og oppdatere dokumentasjon fortløpende.

11.3.3 Teknisk gjeld og vedlikehold

Vi forsøkte å begrense teknisk gjeld ved å prioritere ren kode, enhetstester og dokumentasjon fremfor raske, midlertidige løsninger. Dette gjør at systemet lettere kan videreutvikles etter prosjektets slutt, både av oss selv og eventuelle nye utviklere. Vi valgte også å bevisst nedprioritere noen nice-to-have-funksjoner til fordel for robust testing, stabilitet og god dokumentasjon.

11.3.4 Teamutvikling

Prosjektet bidro til å utvikle oss fra en gruppe studenter med ulik bakgrunn til et mer samkjørt utviklingsteam. Gjennom bruk av Scrum, felles verktøy og tydelige roller opplevde vi økt eierskap, bedre kommunikasjon og høyere kvalitet på leveransene. Rollene ble fordelt etter kompetanse: Philip som teknisk leder, Erica som Product Owner, Simen som frontend-spesialist, Jimmy som backend-spesialist, og Mohamed som DevOps- og testansvarlig.

Kapittel 12

Arkitektur

12.1 Systemarkitektur

Systemet er utviklet som en moderne webapplikasjon som består av tre primære komponenter: et React-basert brukergrensesnitt, en applikasjonsserver implementert med Spring boot og en PostgreSQL-database. Arkitekturen følger en tre lagsmodell med tydelige separasjoner mellom presentasjonslag, forretnings logikk og datalagring. Inndelingen legger en grunnlag for en strukturert og vedlike holdbar systemutviklingsprosess.

12.1.1 Arkitektur valg

Applikasjonen bygger på prinsipper om lagdeling og modularitet. Hver av disse systemkomponentene er tildelt av et avgrenset ansvarsområde, dette reduserer kompleksitet og muliggjør uavhengig av utvikling og testing. Valget av en klient-server-arkitektur er ønsket om å separere brukergrensesnitt fra forretnings logikk. Denne separasjonen gjør det enklere å videreutvikle systemet over tid.

12.2 Systemkomponenter

12.2.1 Frontend

Frontend er utviklet med React og TypeScript og fungerer som en dynamisk nettside som oppdateres uten å laste hele siden på nytt. Arkitekturen er delt inn i pages og komponenter som kan gjenbrukes. Navigasjonen håndteres av React Router, og kommunikasjonen med serveren skjer gjennom vanlige HTTP-forespørsler. Dette gir et raskt og responsivt grensesnitt, og gjør det enklere å strukturere funksjonaliteten på en oversiktlig måte.

12.2.2 Backend

Backend er bygget i Spring Boot, organisert i moduler som hver representerer en del av systemet. For eksempel innlogging, ruteinformasjon, betaling, billetter og administrasjon. Fordelingen i modulene gjør systemet vårt mer ryddig og enklere å videreutvikle og teste funksjoner for seg. Serveren vil sørge for at regler følges, forespørsler behandles riktig, og data hentes og lagres på en forutsigbar måte.

12.2.3 Datalagring

Alt av informasjon lagres i en relasjonsdatabase (PostgreSQL). Her vil vi finne brukere, billetter, ruter, stoppesteder og avgangstider. Datamodellen er organisert på en måte som unngår unødvendig duplikat og gjør det enkelt å hente ut data fra systemet. Endringer i databasen håndteres med en Flyway-skript, som gjør at alle endringer kan spores og gjenskapes.

12.2.4 API-Design og kommunikasjon

Frontend og backend jobber sammen gjennom et REST-basert API. Dette innebærer at nettleseren sender forespørsler til serveren for å få informasjon og utfører handlinger som å søke etter ruter, kjøpe billetter og logge inn.

12.3 Avhengigheter og funksjonalitet

12.3.1 Rutedata

Rutedata for busslinjer i applikasjonen er forhåndsdefinerte og ligger lagret i databasen. Dette er valgt for å unngå avhengighet til eksterne leverandører i utviklingsfasen.

12.3.2 Sanntids informasjon

Sanntidsinformasjon i applikasjonen er implementert som stub-funksjonalitet med mockdata. RealtimeController tilbyr API-enderpunkter for bussposisjoner og avganger, mens RouteController støtter en delay-parameter for manuell simulering av forsinkelser. Database-struktur for lagring av forsinkelser er etablert. Dette er valgt for å demonstrere funksjonalitet uten avhengighet til eksterne sanntidskilder i utviklingsfasen. Arkitekturen er lagt til rette for fremtidig integrasjon.

12.3.3 betaling

Betaling funksjonen er også simulert. Brukeren kan registrere betalings metoder, men ingen transaksjoner gjennomføres. Dette vil gi en god brukeropplevelse uten krav til betalings leverandør.

12.4 Begrensinger og antagelser

12.4.1 Tekniske begrensinger

Rutedata oppdateres ikke automatisk fra eksterne API-er, betalinger og sanntids forsinkelser er simulert. Kartbasert funksjonalitet bruker forhåndlagrede posisjoner vi har lagt til. Backend kjører som en samlet applikasjon, og ikke på flere servere.

12.4.2 Funksjonelle antagelser

Brukere vil bli identifisert med unik e-postadresse, systemet dekker kun Østfold området. Billettvalidering skjer digitalt uten en fysisk skanner og administrator har tilgang til utvidet funksjonalitet.

12.5 Deployment og arbeidsflyt

Prosjektet benytter GitHub Actions til å automatisere bygging og testing av applikasjonen, som en del av vår CI/CD-pipeline. Deployment gjennomføres på en kontrollert måte for å redusere risikoen for feil, og denne strukturerte arbeidsflyten bidrar til en mer stabil og forutsigbar utviklingsprosess.

12.6 Oppsummering

Arkitekturen vår er utformet for å være ryddig, forståelig og enkel å videre utvikle. Selv om enkelte funksjoner er simulert og har med dummydata, er systemet strukturert slik at det enkelt kan utvides med ekte data og tjenester. Inndelingen mellom frontend, backend og database gir en god struktur for videre arbeid.

Kapittel 13

Prototype

13.1 Prototype-rapport: Implementert funksjonalitet

13.1.1 Implementerte krav fra kapittel 4

Fullstendig implementerte MVP-krav

F01 – Brukerautentisering og roller (Fullført) Systemet har et komplett autentiseringsoppsett basert på JWT og Spring Security. Løsningen støtter registrering, innlogging, roller, sikre API-endepunkter, passordhashing og funksjon for glemt passord. Frontend håndterer token via localStorage, mens roller lagres i databasetabellene `role` og `user_role`.

F02 – Ruteplanlegging (Fullført) Ruteplanleggingen er implementert med dedikerte komponenter i frontend og backend. Brukeren kan søke mellom destinasjoner, få rutealternativer og se estimert reisetid. Kartvisning er løst med Leaflet. Rutedata er lagret i `route`, `stop` og `route_stop` som statisk seed-data for Østfold-regionen.

F03 – Oversikt over avganger (Fullført) Systemet viser en strukturert oversikt over bussavganger med sortering, filtrering og detaljert visning av tidspunkt, rutenummer og holdeplasser. Backend henter informasjon fra `departure`-tabellen, og UI-komponenter i frontend presenterer data på en konsistent måte.

F04 – Digital billettering (Fullført) Brukere kan kjøpe enkelt- og periodebilletter. Systemet genererer QR-koder, viser aktive billetter og lagrer kjøpshistorikk. Backend håndterer logikk for billettstatus og betalingstyper. Betaling er simulert og ikke koblet til eksterne leverandører.

F05 – Grunnleggende sanntidsinformasjon (Delvis) Det finnes databaseoppsett, API-endepunkt og visningskomponenter for sanntidsinformasjon. Data er foreløpig dummy-generert og ikke koblet til eksterne API-er.

F06 – Administrasjonsgrensesnitt (Fullført) Admin-delen gir oversikt over brukere, roller og systeminnstillinger. Backend håndteres av `AdminController`, og tilgangsstyring baseres på RBAC.

F07 – CI/CD-pipeline (Fullført) Prosjektet inkluderer GitHub Actions, pre-commit hooks, automatisert bygg, testing og deployment til produksjonsserver.

Utvidede funksjonelle krav

F08 – Flerspråklig støtte (Fullført) Systemet inneholder full i18n-støtte gjennom `react-i18next`. Norsk og engelsk er implementert med oversettelsesfiler, språkvelger, automatisk språkdeteksjon og persistering av språkvalg.

F09–F12 – Ikke implementert Push-varsler, offline-støtte, avansert sanntidsinformasjon og preferansebasert ruteplanlegging er ikke inkludert i prototypen.

13.2 Hvordan prototypen startes og brukes

Brukeren kan enten benytte produksjonsversjonen på <https://itstud.hiof.no/philipag/app/> eller kjøre løsningen lokalt. Produksjonsversjonen gir tilgang til registrering, innlogging, rutesøk, avganger, billettkjøp og profiladministrasjon. Instruksjoner for lokal kjøring beskriver oppstart av frontend med Node.js og backend med Java 17.

13.3 Testing

Systemet har omfattende testdekning for frontend og backend. Frontend-testene dekker komponentrendering, interaksjoner, routing, API-mocking og linting. Backend-testene dekker service-lag, repository, kontrollere, autentisering og sikkerhet. Pre-commit hooks sikrer at tester kjøres før hver commit.

13.4 Kjente svakheter og begrensninger

13.4.1 Datakilder og integrasjoner

Rutedata er statisk og ikke koblet til eksterne API-er. Sanntidsdata og betaling er simulert. Systemet reflekterer derfor ikke faktiske ruteendringer eller reelle transaksjoner.

13.4.2 Tekniske begrensninger

Prototypen er monolittisk og ikke optimalisert for horisontal skalering. Sikkerhetskonnfigurasjonen er ikke produksjonsklar og mangler blant annet sikre nøkkelhåndteringsrutiner. Løsningen har heller ingen støtte for offline-modus.

13.4.3 Funksjonelle begrensninger

Billetter kan ikke valideres i praksis. Dekningsområdet er begrenset til Østfold-regionen, og administrasjonsgrensesnittet mangler avanserte funksjoner som bulk-import av rutedata.

13.5 Sammenligning: Planlagt vs. implementert

Prototypen oppfyller seks av syv MVP-krav fullt ut. Ett krav (F05) er delvis implementert. I tillegg er flerspråklig støtte (F08) gjennomført tidligere enn planlagt. Dette gir en måloppnåelse på rundt 85–90%. Flere ikke-planlagte sider og funksjoner, som utvidet profilhåndtering og innstillingssider, er også inkludert.

Beskriv hva som faktisk er implementert i MVP-en.

- Hvilke krav fra kapittel 4 er implementert
- Hvordan prototypen kan startes og brukes (enkle steg for ikke-tekniske brukere)
- Hvordan testene kjøres, og hva de tester

Ta også med kjente svakheter eller begrensninger ved prototypen.

Kapittel 14

Testing og validering

Prototypen har blitt testet gjennom enhetstester (unit tests) som fokuserer på de mest kritiske funksjonene i systemet. Testene er implementert ved bruk av JUnit 5 og Mockito, som er industristandarder for testing av Java-applikasjoner. Dette kapittelet beskriver hvordan prototypen er testet, hvilke krav som er verifisert, og hvordan systemet håndterer feil og uventede situasjoner.

14.1 Kjøring av tester i IntelliJ IDEA

Testene kan kjøres på flere måter i IntelliJ IDEA, som gir fleksibilitet i utviklingsarbeidsflyten. Den mest direkte metoden er å høyreklikke på testklassen eller en spesifikk testmetode i prosjektnavigatoren og velge Run 'TestClassName' eller Run 'testMethodName()' fra kontekstmenyen. Alternativt kan testene kjøres ved å klikke på grønn pil-ikonet som vises ved siden av testklassenavnet eller testmetodenavnet i editoren, noe som gir rask tilgang til testkjøring.

For å kjøre alle tester i prosjektet, kan man utnytte Maven-integrasjonen i IntelliJ IDEA ved å åpne Maven-toolvinduet (View → Tool Windows → Maven) og kjøre målene test eller verify. Dette sikrer at alle tester kjøres i riktig rekkefølge og med korrekt klassepåstigningskonfigurasjon. Alternativt kan dette gjøres via innebygd terminal i IntelliJ IDEA ved å navigere til Backend-mappen og kjøre kommandoen mvn test, som utfører hele testfasen i Maven build lifecycle.

IntelliJ IDEA gir også mulighet for å kjøre tester med kodedekning (code coverage) ved å høyreklikke på testklassen og velge Run 'TestClassName' with Coverage". Dette gir kvantitativ innsikt i hvilke deler av koden som er dekket av testene, og hvilke deler som mangler testdekning. Coverage-rapporten vises i et eget vindu og kan eksporteres for dokumentasjon eller kvalitetssikring.

Testresultatene vises i Runvinduet i IntelliJ IDEA, hvor man kan se hvilke tester som passerte, hvilke som feilet, og detaljert informasjon om eventuelle feilmeldinger og stack traces. Dette gjør det enkelt å identifisere og feilsøke problemer i koden, og gir rask tilbakemelding om kodekvalitet under utviklingsprosessen.

Merk: For Java 25-kompatibilitet er det nødvendig å konfigurere Maven Surefire-plugin med `net.bytebuddy.experimental=true` system property for å støtte Mockito's bytecode-manipulering. Dette er konfigurert i `pom.xml` og sikrer at testene kan kjøres med nyere Java-versjoner.

14.2 Teststrategi og tilnærming

For å sikre høy kvalitet og pålitelighet i prototypen, har vi implementert en omfattende testsuite som fokuserer på de mest kritiske komponentene i systemet. Teststrategien er basert på risikobasert testing, hvor funksjoner med høyest risiko for feil og størst innvirkning på systemets sikkerhet og funksjonalitet prioriteres. Denne tilnærmingen sikrer effektiv ressursbruk og fokuserer på områder hvor feil vil ha størst konsekvens.

Testene er organisert i tre hovedkategorier: sikkerhetstester for autentisering og autorisering, forretningslogikk-tester for kjernefunksjonalitet som billettkjøp, og integrasjonstester for interaksjon mellom komponenter. Denne kategoriseringen sikrer systematisk dekning av systemets kritiske aspekter og følger best practices innen software testing.

14.3 Hvilke krav som er testet

Testene dekker følgende kritiske funksjonelle krav:

14.3.1 Autentiseringskrav

Brukerregistrering representerer en kritisk funksjon som krever at systemet kan registrere nye brukere med unik e-postadresse. Implementeringen i `AuthService` bruker dependency injection av `UserRepository` og `PasswordEncoder` for å sikre løs kobling og testbarhet. Metoden `register()` implementerer defensive programming ved først å sjekke om e-postadressen allerede eksisterer:

```
public boolean register(String email, String password, String fullName) {
    if (userRepository.existsByEmail(email)) {
        return false;
    }

    User user = new User();
    user.setEmail(email);
    user.setPasswordHash(passwordEncoder.encode(password));
    user.setFullName(fullName);
    user.setIsActive(true);

    userRepository.save(user);
    return true;
}
```

Testene verifiserer at duplikate e-postadresser avvises korrekt, at passord hashes korrekt før lagring ved bruk av Spring Security's PasswordEncoder, at nye brukere opprettes med aktiv status, og at brukerdata lagres korrekt i databasen. Dette sikrer dataintegritet og forhindrer sikkerhetsrisikoer knyttet til duplikate registreringer.

Innlogging krever at systemet validerer brukerlegitimasjoner på en sikker måte. Implementeringen bruker Optional.orElse(null) for null-sikker håndtering av ikke-eksisterende brukere, og sjekker brukerens aktive status før passordvalidering:

```
public boolean login(String email, String password) {
    User user = userRepository.findByEmail(email).orElse(null);
    if (user == null || !user.getIsActive()) {
        return false;
    }

    return passwordEncoder.matches(password, user.getPasswordHash());
}
```

Testene verifiserer at korrekte legitimasjoner gir tilgang til systemet, at feil passord avvises uten å avsløre om brukeren eksisterer (sikkerhetstiltak mot brukerenummerering), at inaktive brukere ikke kan logge inn selv med korrekt passord, og at eksisterende brukere kan hentes fra databasen på en pålitelig måte.

JWT-token generering og validering er fundamentalt for systemets sikkerhet. Implementeringen bruker JJWT-biblioteket for å generere tokens i henhold til JWT-standard (RFC 7519). Token-generering bruker HMAC-SHA256 for signering og setter utløpstid til 24 timer:

```
public String generateToken(String username) {
    Map<String, Object> claims = new HashMap<>();
    return createToken(claims, username);
}

private String createToken(Map<String, Object> claims, String subject) {
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(subject)
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + JWT_EXPIRATION))
        .signWith(getSigningKey(), SignatureAlgorithm.HS256)
        .compact();
}
```

Token-validering implementerer flere lag med sjekk for å sikre autentiseringsintegritet. Systemet håndterer utløpte tokens på en elegant måte ved å fange ExpiredJwtException og ekstrahere informasjon fra tokenets claims selv etter utløp:

```
public String extractUsername(String token) {
    try {
        return extractClaim(token, Claims::getSubject);
    } catch (ExpiredJwtException e) {
        // Even if expired, we can extract the username from the expired token
        return e.getClaims().getSubject();
    }
}
```



```

    }
}

public Boolean validateToken(String token, String username) {
    try {
        final String extractedUsername = extractUsername(token);
        return (extractedUsername.equals(username) && !isTokenExpired(token));
    } catch (Exception e) {
        // If any exception occurs, the token is invalid
        return false;
    }
}
}

```

Testene verifiserer at tokens genereres med korrekt struktur i henhold til JWT-standarden, at tokens inneholder korrekt brukernavn, at tokens har utløpstid på 24 timer, og at utløpte tokens avvises korrekt uten å kaste unntak. Videre verifiseres det at tokens for feil bruker avvises, og at ugyldige eller manipulererte tokens håndteres gracefully ved å returnere false i stedet for å kaste unntak, noe som gir bedre brukeropplevelse og er avgjørende for å forhindre uautorisert tilgang.

14.3.2 Forretningslogikk-krav

Billett kjøp representerer kjernefunksjonaliteten i systemet og må håndteres på en pålitelig måte. Implementeringen bruker Spring's @Transactional annotasjon for å sikre atomiskhet i database-operasjoner. Metoden purchaseTicket() håndterer både autentiserte brukere og gjester ved å bruke en standard gjestebruker-ID når userId er null:

```

@Transactional
public Ticket purchaseTicket(BuyRequest request, Long userId) {
    if (userId == null) {
        userId = 1L; // Default guest user ID
    }

    // Determine fare type with fallback logic
    Ticket.FareType fareType = Ticket.FareType.ADULT;
    if (request.getSeniors() != null && request.getSeniors() > 0) {
        fareType = Ticket.FareType.SENIOR;
    } else if (request.getFareType() != null) {
        try {
            fareType = Ticket.FareType.valueOf(request.getFareType());
        } catch (IllegalArgumentException e) {
            fareType = Ticket.FareType.ADULT;
        }
    }

    String qrCodeHash = generateQrCodeHash();
    // ... rest of implementation
}

```

Testene verifiserer at billetter kan kjøpes både av autentiserte brukere og av gjester uten innlogging, noe som gir fleksibilitet i brukeropplevelsen. Videre verifiseres det at korrekt billetttype beregnes basert på input (voksen, student eller senior), at pris settes korrekt til 40kr, og at unike QR-koder genereres for hver billett. Billettstatus settes til NEW ved opprettelse, og transaksjoner håndteres atomisk slik at enten fullføres hele operasjonen eller ingenting, noe som sikrer dataintegritet selv ved feil.

QR-kode generering må produsere unike og sikre identifikatorer for billetter. Implementeringen bruker SHA-256 hashing av UUID for å generere kryptografisk sikre identifikatorer:

```
private String generateQrCodeHash() {
    String uuid = UUID.randomUUID().toString();
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(uuid.getBytes(StandardCharsets.UTF_8));
        StringBuilder hexString = new StringBuilder();
        for (byte b : hash) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    } catch (Exception e) {
        // Fallback to UUID if hashing fails
        return uuid.replace("-", "");
    }
}
```

Testene verifiserer at hver QR-kode er unik, at QR-koder genereres i korrekt format som en 64-tegns hex-streng, og at QR-koder er kryptografisk sikre og ikke forutsigbare. Dette er avgjørende for å forhindre billettsvindel og sikre at hver billett kan valideres på en pålitelig måte ved scanning.

14.4 Hvordan testene verifiserer funksjonaliteten

Testene bruker en kombinasjon av enhetstesting og mocking for å isolere og verifisere hver komponent individuelt. Denne tilnærmingen følger prinsippet om dependency injection og inversion of control, hvor avhengigheter mockes ved bruk av Mockito's @Mock og @InjectMocks annotasjoner. Dette gir flere betydelige fordeler i form av raskere testkjøring, økt pålitelighet gjennom eliminering av avhengigheter til eksterne systemer, raskere feillokalisering til spesifikke funksjoner, og mulighet for å kjøre testene i hvilken som helst rekkefølge.

14.4.1 Isolert testing

Ved å bruke Mockito for å mocke avhengigheter som databasetilgang og eksterne tjenester, kan vi teste hver funksjon isolert uten faktisk database-tilgang eller avhengighet av eksterne systemer. Dette følger prinsippet om unit testing hvor hver enhet testes i isolasjon. Testene bruker Mockito's `when().thenReturn()` syntaks for å definere mock-oppførsel, og `verify()` for å bekrefte at metoder er kalt med forventede parametere. Dette gjør testene både raskere og mer pålitelige, samtidig som feil kan lokaliseres raskere til spesifikke funksjoner.

14.4.2 Testdekning av edge cases

Testene dekker ikke bare "happy path"scenarier, men også edge cases og feilsituasjoner. Dette inkluderer håndtering av null-verdier i input ved bruk av `Optional.orElse(null)` pattern, ugyldige data som ugyldig billetttype håndtert gjennom try-catch-blokker, manglende brukere eller ressurser håndtert gjennom defensive null-sjekker, avvisning av duplikate e-postadresser verifisert gjennom `existsByEmail()` sjekk, og korrekt håndtering av utløpte tokens verifisert gjennom dato-sammenligning. Denne omfattende testdekningen sikrer at systemet oppfører seg korrekt også under uventede forhold.

14.4.3 Verifisering av forretningsregler

Testene verifiserer at forretningsreglene implementeres korrekt. Billetttype-logikken sørger for at hvis en forespørsel inneholder seniorer, settes billetttypen til SENIOR, ellers brukes den spesifiserte typen eller ADULT som standard. Dette implementeres gjennom en if-else struktur med fallback-mekanisme. Prisberegningen verifiseres til å være fast på 40kr for alle billetter, implementert som en konstant verdi. Transaksjonshåndtering sikres gjennom bruk av Spring's `@Transactional` annotasjon som garanterer ACID-egenskaper (Atomicity, Consistency, Isolation, Durability), og standardverdier brukes når input mangler, for eksempel brukes standardverdien 1L hvis `departureId` ikke er spesifisert, implementert gjennom ternary operator.

14.5 Hvordan systemet håndterer feil og uventede situasjoner

Systemet er designet for å håndtere feil og uventede situasjoner på flere nivåer gjennom en kombinasjon av validering, feilhåndtering og defensive programming-teknikker.

14.5.1 Validering av input

Inputvalidering skjer på flere nivåer for å sikre datakvalitet og forhindre feil. E-postvalidering sjekker om e-postadressen allerede eksisterer før registrering gjennom `userRepository.existsByEmail(email)`, og returnerer false hvis brukeren allerede finnes, noe som forhindrer duplikate registreringer og sikrer referanseintegritet. Brukerstatus sjekkes før innlogging gjennom `user.getIsActive()`, slik at inaktive

brukere ikke får tilgang til systemet. Passordvalidering implementeres gjennom Spring Security's `PasswordEncoder.matches()` metode, som gir sikker passordvalidering som er resistent mot timing-angrep og andre sikkerhetstrusler ved bruk av konstant-tids sammenligning.

14.5.2 Feilhåndtering i autentisering

Autentiseringssystemet håndterer feilsituasjoner på en forutsigbar måte ved å bruke defensive programming-teknikker. Hvis en bruker ikke finnes ved innlogging, håndteres dette gjennom `Optional.orElse(null)` pattern, og false returneres uten å kaste unntak, noe som gir en ren feilhåndtering og forhindrer unødvendige exception overhead. Ved feil passord returnerer systemet false uten å avsløre om brukeren eksisterer, noe som er et viktig sikkerhetstiltak mot brukerenummerering (user enumeration attacks). Inaktive brukere får ikke tilgang til systemet, selv med korrekt passord, noe som gir administratorene kontroll over brukertilgang gjennom `isActive` flagget.

14.5.3 Feilhåndtering i JWT-tokens

JWT-token håndtering implementerer flere lag med validering for å sikre autentiseringsintegritet. Systemet håndterer utløpte tokens elegant ved å fange `ExpiredJwtException` og ekstrahere informasjon fra tokenets claims selv etter utløp, noe som gir mulighet for å validere tokenet uten å kaste unntak. Dette implementeres gjennom try-catch-blokker i `extractUsername()` og `extractExpiration()` metoder som fanger `ExpiredJwtException` og henter informasjon fra `e.getClaims()`. Systemet sjekker token-utløpstid gjennom `isTokenExpired()` som returnerer true hvis tokenet er utløpt, og avviser utløpte tokens på en forutsigbar måte. Ugyldige eller manipulerede tokens håndteres ved å fange alle unntak i `validateToken()` og returnere false, slik at klienten får tydelig tilbakemelding uten unntak. Tokens som ikke matcher det forventede brukernavnet avvises gjennom `extractedUsername.equals(username)` sjekk, og null-tokens eller ugyldige token-formater håndteres gracefully ved å returnere false, noe som forhindrer null pointer exceptions og gir forutsigbar feilhåndtering.

14.5.4 Feilhåndtering i billettkjøp

Billettkjøpsprosessen er designet for å håndtere manglende eller ugyldig data på en robust måte gjennom defensive programming og graceful degradation. Hvis en bruker-ID ikke eksisterer, fortsetter systemet med gjestekjøp ved å bruke standard gjestebruker-ID (1L), noe som gir fleksibilitet i brukeropplevelsen. Hvis en ugyldig billetttype spesifiseres, faller systemet tilbake til ADULT som standard gjennom en try-catch-blokk som fanger `IllegalArgumentException`. Hvis `departureId` ikke er spesifisert, brukes standardverdien 1L gjennom ternary operator. QR-kode generering har en fallback-mekanisme hvor systemet faller tilbake til UUID-basert generering hvis SHA-256 hashing feiler, implementert gjennom exception handling. Database-feil håndteres gjennom Spring's `@Transactional` annotasjon som sikrer at hvis noe går galt, rulles alle endringer tilbake gjennom rollback-mekanismen, og databasen forblir i konsistent tilstand i henhold til ACID-prinsippene.

14.5.5 Defensive programming

Systemet implementerer defensive programming-teknikker for å forhindre feil og håndtere uventede situasjoner. Null-sjekker utføres før bruk av verdier gjennom eksplisitte null-sjekker (`if (user == null)`) og Optional pattern, og null-verdier håndteres på en forutsigbar måte uten å kaste unødvendige unntak. Systemet bruker fornuftige standardverdier når input mangler gjennom ternary operators og default-verdier, noe som gir robusthet mot ufullstendig data. Kritiske operasjoner er wrappet i try-catch-blokker for å forhindre uventede krasj, spesielt i QR-kode generering hvor `NoSuchAlgorithmException` kan oppstå. Database-operasjoner bruker Spring's `@Transactional` annotasjon for å sikre dataintegritet selv ved feil gjennom automatisk rollback-mekanisme.

14.6 Testresultater og kvalitetssikring

Alle testene er implementert og verifiserer at de kritiske funksjonene oppfører seg som forventet. Testene dekker 18 testscenarier for autentiseringstjenesten (`AuthService`), 12 testscenarier for JWT-tjenesten (`JwtService`), og 11 testscenarier for billett kjøpstjenesten (`TicketPurchaseService`). Dette gir totalt 41 testscenarier som verifiserer de mest kritiske funksjonene i systemet.

14.7 Konklusjon

Gjennom omfattende enhetstesting har vi verifisert at prototypen håndterer de kritiske funksjonene korrekt, inkludert autentisering, autorisering og billett kjøp. Testene sikrer at systemet beskytter mot vanlige sikkerhetsrisikoer som duplikate registreringer og ugyldige tokens, håndterer feil og uventede situasjoner på en forutsigbar måte gjennom defensive programming, opprettholder dataintegritet gjennom ACID-transaksjoner, genererer unike og kryptografisk sikre QR-koder for billetter ved bruk av SHA-256 hashing, og validerer input korrekt gjennom bruk av defensive programming-teknikker og null-sikker håndtering.

Testene fungerer som en viktig kvalitetssikring før kompilering og deployering, og kan kjøres automatisk som del av en CI/CD-pipeline (Continuous Integration/Continuous Deployment) for å fange opp feil tidlig i utviklingsprosessen. Dette gir høy tillit til at systemet oppfører seg korrekt under produksjonsforhold og håndterer edge cases og feilsituasjoner på en robust måte gjennom omfattende testdekning av både happy path og edge case scenarier.

Kapittel 15

Bruken av kunstig intelligens

Vi har brukt kunstig intelligens som et støttende verktøy i utviklingsprosessen. Målet har vært å styrke egen forståelse, redusere tidsbruk ved feilsøking og forbedre kvaliteten på arbeidet. KI har ikke blitt brukt til å produsere hele løsninger eller utvikle sentrale deler av systemet, men heller som et hjelpe middel til vårt eget arbeid.

En sentral bruk av KI har vært å feilsøking. Når vi har stått fast på grunn av uklare feilemeldinger, har vi brukt KI til å tolke problemet og foreslå mulige løsninger. Dette har bidratt til en mer effektiv flyt og gjort det enklere å lære av feilene.

Vi har også brukt KI for å få en strukturert og mer ryddig kode. Dette gjelder spesielt når større kodeblokker blir uoversiktlige. Vi har for eksempel ønsket forslag til bedre organisering uten at det gikk på bekostning av vår applikasjon.

Videre har KI blitt brukt til å forstå hvordan man kan utforme en autentisering og brukerhåndtering i et moderne websystem. Her har kunstig intelligens blitt brukt til å forklare prinsipper og gi eksempler på syntakser, ikke til å generere ferdig kode. Dette har styrket den tekniske forståelsen og gitt oss et godt grunnlag for å lage vår egen løsning.

Samlet sett har vi brukt kunstig intelligens på en ansvarlig og bevisst måte. Verktøyet har blitt brukt til læring, feilsøking og konseptuell veiledning, og ikke som en erstatning for utviklingsarbeidet. Dette har gjort det mulig å få støtte der det trengtes samtidig som prosjektet i hovedsak er utviklet gjennom fagkunnskap og egen innsats.

15.1 Verktøy brukt i prosjektet

15.1.1 ChatGPT

ChatGPT ble brukt som et støtteverktøy i utviklingsprosessen [2]. Verktøyet bidro særlig ved feilsøking, innhenting av forklaringer på tekniske problemstillinger, planlegging av noen arbeidsoppgaver og

avklaringer rundt noen generelle spørsmål i prosjektet. Bruken var primært løsningsorientert og fungerte som et hjelpemiddel til egen kompetanse.

15.1.2 Claude Sonnet

Claude Sonnet ble benyttet spesielt mot konseptuell veiledning og kvalitetsikring av både tekst og struktur [1]. Den ble brukt ved feilsøking og vurdering av faglige sammenhenger, spesielt der det var ekstra behov for presise forklaringer.

15.1.3 Prettier

Prettier ble brukt til automatisk formatering av kode for å sikre en konsistent og lesbar kodebase [3]. Dette bidro til et bedre samarbeid i utviklings teamet og redusert tid brukt på justering av formatering.

Kapittel 16

Oppsummering og konklusjon

I dette prosjektet har vi utarbeidet en kravspesifikasjon for en ny buss-app som skal være et mer brukervennlig og funksjonelt alternativ til dagens løsning, Østfold kollektiv. Gjennom analyser av dagens system, samt utvikling av personas, scenarier og brukerhistorier, har vi identifisert konkrete behov hos brukerne. Dette har lagt grunnlaget for tydelig definerte funksjonelle og ikke-funksjonelle krav, samt en teknisk spesifikasjon for en Minimum Viable Product (MVP).

Løsningen vi foreslår gir oppdragsgiver en moderne app med forbedret ruteplanlegging, sanntidsinformasjon, digital billettering og et administrasjonsgrensesnitt. Disse funksjonene vil øke brukervennligheten, tilgjengeligheten og påliteligheten for kundene, samtidig som arbeidsprosesser for administratorer og utviklere forenkles. For oppdragsgiver innebærer dette en bedre tjenesteopplevelse for sluttbrukerne og en mer effektiv drift.

Videreutviklingsmuligheter inkluderer flerspråklig støtte, push-varsler, offline-funksjonalitet og integrasjon mot eksterne sanntidsdatakilder. Slike utvidelser vil ytterligere styrke appens verdi, gjøre den mer attraktiv for et bredere spekter av brukere, og bidra til økt konkurransekraft for kollektivtilbudet. Prosjektet danner dermed et solid grunnlag for en løsning som kan videreutvikles til et fullverdig, produksjonsklart system.

Referanseliste

- [1] Anthropic. Claude Sonnet – large language model. <https://claude.ai>, 2025. Brukt som støtteverktøy for idéutforming, kvalitetssikring av tekst og tekniske forklaringer. Tilgjengelig per 21. november 2025.
- [2] OpenAI. ChatGPT – large language model. <https://chat.openai.com>, 2025. Brukt som støtteverktøy for feilsøking, planlegging og teknisk veiledning. Tilgjengelig per 21. november 2025.
- [3] Prettier. Prettier – code formatter. <https://prettier.io>, 2025. Brukt til automatisk formatering av kode. Tilgjengelig per 21. november 2025.
- [4] Ian Sommerville. *Engineering Software Products: An Introduction to Modern Software Engineering*. Pearson Education Limited, Harlow, England, global edition edition, 2021.

